Insecure by Design

A Hands-On Learning Platform for Web Security Training

Final Report

CSIS 4495-003 | Applied Research Project

Winter 2026

Instructor: Arasanipalai Kandhadai, Padmapriya

Allwyn Mascarenhas | 300381351 | Team Leader

Thilan Udahage Don | 300398798

Table of Contents

1. Introduction 6

2. Summary of the Research Project 7

3. Changes to the Proposal 8

3.1 Shift from Demo-Based to Challenge-Based Architecture 8

3.2 Addition of the 'Why It Worked' Explainer 8

3.3 Addition of the User Profile Progress Tracker 9

3.4 Refinement of the XSS Vulnerability Module 9

3.5 Revised Timeline 9

3.6 Shift to Collaborative, Iterative Development 10

3.7 Cloud Deployment and Hosting 10

4. Project Completion Timeline 10

4.1 Actual Project Schedule and Milestones 10

4.2 Team Responsibilities 12

4.3 Project Management Chart 13

5. Implemented Features 13

5.1 Vulnerabilities and Exploits 13

5.1.1 SQL Injection 13

5.1.2 Reflected Cross-Site Scripting (XSS) 15

5.1.3 Cross-Site Request Forgery (CSRF) 17

5.1.4 Insecure Direct Object Reference (IDOR) 19

5.1.5 Broken Authentication 20

5.1.6 Insecure File Upload 22

5.2 User Features 25

5.2.1 Why It Worked' Explainer 25

5.2.2 User Profile Progress Tracker 26

5.2.3 Homepage and Platform UI 27

6. Evaluation 29

6.1 Functional Evaluation: User Acceptance Testing (UAT) 29
6.1.1 General System Checks 29
6.1.2 Broken auth 30
6.1.3 SQL INJ 30
6.1.4 XSS Reflected 31
6.1.5 Insecure file upload 31
6.1.6 IDOR 32
6.1.7 CSRF 32
6.1.8 XP and Progression 33
6.2 Educational Evaluation: User Testing Methodology 33
6.3 Educational Evaluation: Survey Results 34
6.4 Design and Implementation Changes Driven by Evaluation 37
7. Reflections and Discussion 37
8. AI Use Section 38
8.1 Table of AI Tools and Specific Use 38
8.2 Appendix - Prompt History Summary 39
9. Work Logs 40
9.1 Thilan Udahage Don 40
9.2 Allwyn Mascarenhas 44
10. Concluding Remarks 49
11. References 50
Appendix A: Installation Guide 51
Appendix B: User Guide 52
Getting Started 52
Completing a Challenge 52
Tracking Your Progress 52
Challenge Modules and Accepted Payloads 52
OWASP Reference 52

Table of Figures

Figure 1 - Kanban board snapshot reflecting the agile project management workflow, task assignments, and completion status across the development lifecycle. 13
Figure 2 - The SQL Injection module interface, displaying the progressive challenge structure and the Level 1 input form. 14
Figure 3 - Backend validation logic for the Level 1 SQL Injection challenge, demonstrating the execution of a user-supplied payload directly against the database to evaluate bypass success. 15
Figure 4 - Reflected XSS Level 2 interface showing a successful event-handler filter bypass and the dynamic reveal of the 'Why It Worked' explainer. 16
Figure 5 - Backend validation logic in XssChallengeService simulating a regex-based script tag filter to evaluate the learner's ability to utilize alternative execution vectors. 17
Figure 6 - he CSRF module interface, prompting the learner to submit a forged state-changing email update request on behalf of an authenticated victim. 18
Figure 7 - Backend implementation of the CSRF challenges, depicting an intentionally unprotected state-changing endpoint alongside a trivially guessable static token validation check. 18
Figure 8 - The IDOR challenge interface showing the successful manipulation of a predictable profile ID parameter to access unauthorized user data. 19
Figure 9 - Backend logic for the IDOR module simulating a user profile database query that fundamentally lacks resource ownership authorization checks. 20
Figure 10 - The Broken Authentication module interface demonstrating the successful exploitation of hardcoded default administrative credentials. 21
Figure 11 - Backend validation logic evaluating predictable session tokens, including Base64 decoding attempts and sequential ID pattern matching. 22
Figure 12 - Insecure File Upload module interface showing a successful MIME type spoofing attack and the generation of a direct execution link. 23
Figure 13 - File Upload — Level 2 checking extension looks like image while content contains script 24
Figure 14 - The 'Why It Worked' Explainer dynamically revealed after a successful exploit, detailing the vulnerable code pattern and industry-standard remediation strategy. 25
Figure 15 - Client-side JavaScript logic responsible for dynamically rendering the contextual remediation advice and code snippets upon a successful AJAX validation response. 25
Figure 16 - The User Profile dashboard utilizing Bootstrap badges and progress bars to display real-time chronological progression and earned module achievements. 26

Figure 17 - The redesigned platform homepage featuring a modernized hero section, platform overview, and modular challenge routing. 27
Figure 18 - The dedicated OWASP Top 10 reference architecture, explicitly mapping the platform's interactive challenges to their corresponding real-world risk categories. 28
Figure 19 - User survey results indicating 100% agreement among participants that the progressive challenge structure significantly aided incremental understanding. 35
Figure 20 - User survey results demonstrating a 90% perfect satisfaction rating for the educational value of the post-challenge 'Why It Worked' explainer. 35
Figure 21 - User survey results identifying Broken Authentication and SQL Injection as the most impactful and educational challenge modules. 36
Figure 22 - User survey results reflecting a high overall satisfaction rating (80% perfect 5/5) for the pedagogical effectiveness of the Insecure by Design platform. 36

1. Introduction

Web application security remains one of the most critical challenges in modern software development. Despite a growing body of training resources, many developers and students continue to enter the workforce without meaningful hands-on exposure to real-world vulnerabilities. The 2024 Fortinet Cybersecurity Skills Gap Report highlights that a significant proportion of security professionals lack practical experience with the very vulnerabilities they are expected to defend against. Existing platforms such as DVWA, WebGoat, and OWASP Juice Shop, while valuable, present limitations: they rely on outdated frameworks, offer limited guidance, and often leave learners without a clear understanding of why an attack succeeded.

This project addresses that gap by developing Insecure by Design, a structured, guided web security training platform built on Spring Boot and Thymeleaf. The platform embeds intentional vulnerabilities — SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Insecure Direct Object Reference (IDOR), Broken Authentication, and unrestricted File Upload — within a progressive challenge framework. Each challenge module presents multiple difficulty levels that unlock sequentially, ensuring learners build understanding incrementally rather than encountering vulnerabilities without context.

The key research question driving this project is: how can a web security training platform be designed to not only expose learners to vulnerabilities, but to actively reinforce their understanding of why those vulnerabilities exist and how they work? Our primary response to this question is the 'Why It Worked' Explainer feature — a novel post-challenge educational section that reveals a detailed explanation of the attack mechanism only after a learner successfully exploits a challenge. While platforms like TryHackMe offer guided learning environments, they lack a dedicated post-challenge educational breakdown. By ensuring that learners engage with the vulnerability directly before receiving the explanation, this feature promotes active rather than passive learning and stands as the unique contribution of our platform.

The project hypothesizes that combining progressive challenge levels with immediate, contextual explanations produces a more effective learning experience than challenge-only or explanation-only approaches. This research assumes that learners possess a foundational understanding of web architecture (such as HTTP requests and basic HTML), allowing them to focus directly on security flaws rather than basic mechanics. Ultimately, the platform is targeted at computer science students and early-career developers seeking structured, hands-on security training within a modern, familiar tech stack, providing the broader educational benefit of bridging the gap between theoretical security knowledge and practical, real-world application.

2. Summary of the Research Project

Insecure by Design is a fully functional web security training platform developed using Java Spring Boot, Thymeleaf, H2 (in-memory database), Bootstrap 5, and Maven. The platform delivers six intentional vulnerability challenge modules, each structured as a progressive, level-based learning experience. Learners interact with an intuitive web interface, submit attack payloads, and receive immediate feedback. Upon successfully completing each level, the 'Why It Worked' Explainer section is revealed, providing a concise technical explanation of the attack mechanism.

The platform includes a user profile progress tracker that displays each learner's completion status across all challenge modules and levels, enabling self-directed progress monitoring. The homepage provides a structured entry point to the platform, with challenge cards for each vulnerability type and a reference OWASP page linking each module to its corresponding OWASP Top 10 category.

The six implemented vulnerability modules, tracked through a shared ChallengeProgress entity to persist learner progress across sessions, include:

- **SQL Injection:** Three progressive levels covering OR-based, numeric, and blind boolean injection.
- **Cross-Site Scripting (XSS):** Demonstrates Reflected XSS through GET-based search parameters.
- **Cross-Site Request Forgery (CSRF):** Highlights the dangers of missing token validation on state-changing requests.
- **Insecure Direct Object Reference (IDOR):** Shows unauthorized data access via predictable numeric IDs.
- **Broken Authentication:** Explores insecure login flows and session fixation.
- **File Upload:** Demonstrates unrestricted file type acceptance.

The project was developed using a highly collaborative, iterative approach. While primary ownership was divided, both team members contributed to the full-stack implementation of several modules:

- **Thilan Udahage Don:** Handled primary implementation for the XSS, CSRF, IDOR, and File Upload modules. Led the UI/UX redesign and progressive challenge architecture, which included refactoring the initial SQL Injection module into a multi-level challenge and updating the User Profile Tracker. Conceived and implemented the central 'Why It Worked' Explainer feature across all vulnerability modules.

- **Allwyn Mascarenhas:** Handled primary implementation for the Broken Authentication module. Led backend Spring Security configurations, database entity design (including the ChallengeProgress model), and developed the initial backend implementation for the SQL Injection and User Profile Progress Tracker modules. Additionally, he contributed crucial backend architecture to the Reflected XSS module by implementing its dedicated controller and Content Security Policy (CSP) configurations.

3. Changes to the Proposal

3.1 Shift from Demo-Based to Challenge-Based Architecture

- **Change:** The original proposal described a demo-based approach in which vulnerability examples would be displayed to learners for passive observation. Following instructor feedback in late February, this approach was replaced with an interactive, challenge-based model where learners actively exploit vulnerabilities to progress.
- **Justification:** Our initial assumption was that observing a vulnerability in a modern framework would be sufficient for learning. However, further research into experiential learning—combined with instructor feedback—highlighted that passive observation does not build practical skills. Changing to a progressive challenge model significantly increased the technical complexity of the project but produced a substantially more engaging and educationally effective platform.

3.2 Addition of the 'Why It Worked' Explainer

- **Change:** The 'Why It Worked' Explainer was not part of the original proposal. It was conceptualized during a team meeting in early March and implemented across all six vulnerability modules, becoming the central novel contribution of the project.
- **Justification:** While evaluating platforms like TryHackMe for guided learning approaches, the team identified a critical missing element in the market: post-challenge educational breakdowns. The 'Why It Worked' Explainer was designed specifically to fill this gap. The decision was made to ensure learners understand the underlying code mechanics of their successful exploits, directly supporting our research goal of bridging theoretical knowledge with practical execution.

3.3 Addition of the User Profile Progress Tracker

- **Change:** The user profile progress tracker was added to the platform. It integrates with a backend `ChallengeProgress` entity to provide learners with a visual summary (via Bootstrap badges) of their completed challenge levels across all modules.
- **Justification:** This feature was added as a direct result of instructor feedback during the March 3 check-in meeting. The justification for this addition was to enhance the user experience by enabling self-directed progress monitoring, allowing learners to easily track which OWASP concepts they had mastered and which they still needed to complete.

3.4 Refinement of the XSS Vulnerability Module

- **Change:** The original proposal planned for a suite of vulnerabilities including SQL Injection, XSS (both Stored and Reflected), Broken Authentication, CSRF, IDOR, and File Upload. During development, the team successfully implemented all proposed modules but made a strategic decision to refine the scope of the XSS module by dropping Stored XSS.
- **Justification:** Rather than implementing both Stored and Reflected XSS at a surface level, the team chose to prioritize depth over breadth. The team focused exclusively on a high-quality, comprehensive implementation of Reflected XSS that demonstrates raw, escaped, and sanitized rendering modes side-by-side. This approach ensured academic clarity and allowed the team to allocate adequate time to fully implement the other proposed vulnerabilities (CSRF, IDOR, File Upload) alongside the 'Why It Worked' Explainer feature.

3.5 Revised Timeline

- **Change:** The original timeline assumed a linear, sequential implementation order for the vulnerability modules. In practice, modules were developed in parallel across the team during the second half of the term.
- **Justification:** The mid-term pivot to a challenge-based architecture in late February required a complete redesign of the SQL Injection module. Because of the time required for this architectural shift, adhering to a sequential timeline would have resulted in an incomplete project. Shifting to parallel, concurrent development allowed the team to complete the expanded scope of the project on schedule.

3.6 Shift to Collaborative, Iterative Development

- **Change:** The original proposal delineated strict frontend and backend roles. As reflected in our progress reports, this shifted to an agile, shared-ownership model where features were developed iteratively by both team members.
- **Justification:** As the project pivoted to a challenge-based architecture, strict role division created bottlenecks. For example, Allwyn developed the initial User Profile Tracker and Reflected XSS controllers in late February, but Thilan iteratively refactored those same controllers and templates in March to integrate the new progressive UI and Explainer features. Furthermore, some modules (like File Upload) were reassigned entirely based on weekly availability. This collaborative overlap, documented in our individual work logs, ensured smooth integration across the platform and allowed both members to gain valuable end-to-end development experience.

3.7 Cloud Deployment and Hosting

- **Change:** The original proposal assumed the platform would be developed, tested, and evaluated exclusively within a local development environment (localhost). The final platform has been containerized and deployed live to Google Cloud Run.
- **Justification:** As the application matured, the team recognized that restricting the platform to local execution created a significant barrier to entry for user evaluation and testing. Deploying the application to a live, publicly hosted URL is justified because it allows seamless user testing without requiring participants to configure Java or Maven. Furthermore, interacting with vulnerabilities on a live web server provides a much more accurate simulation of real-world web application security flaws than a local environment.

4. Project Completion Timeline

4.1 Actual Project Schedule and Milestones

The following timeline reflects the actual development lifecycle of the project from the beginning to the end of the term. While the early weeks followed the original proposal, the mid-term pivot to an interactive challenge-based architecture required agile, concurrent development of the vulnerability modules from Week 7 onward.

Week	Phase	Milestones	Deliverables
Week 1 (Jan 6)	Research	Brainstorm project ideas; Conduct preliminary research on existing platforms (DVWA, WebGoat) and OWASP Top 10.	Interest Survey
Week 2 (Jan 13)	Planning	Finalize project direction; Begin drafting pre-proposal and evaluating Spring Boot security configurations.	
Week 3 (Jan 20)	Planning	Draft project proposal; Finalize technology stack (Spring Boot, Thymeleaf, H2); Create GitHub repository structure.	
Week 4 (Jan 27)	Planning	Submit project proposal; Finalize architecture outline; Refine methodology based on instructor feedback.	Project Proposal
Week 5 (Feb 3)	Setup	Set up local development environment; Initialize Spring Boot skeleton; Configure H2 database and test basic routing.	
Week 6 (Feb 10)	Implementation	Initial backend setup and vulnerability research; Develop early UI templates and initial SQLi implementation.	Progress Report 1
Week 7 (Feb 24)	**Mid-Term Pivot**	Redesign SQLi to a progressive challenge architecture; Implement Reflected XSS endpoints and CSP headers; Record demo.	Midterm Report & Video
Week 8 (Mar 3)	Implementation	Conceptualize 'Why It Worked' Explainer; Develop ChallengeProgress backend entity to persist learner data.	Progress Report 2
Week 9 (Mar 10)	Implementation	Implement Broken Authentication; Create initial User Profile Tracker; Refactor challenge UI across modules.	Progress Report 3

Week 10 (Mar 17)	Implementation	Implement CSRF, IDOR, and File Upload modules; Integrate the 'Why It Worked' Explainer across all challenges.	Progress Report 4
Week 11 (Mar 24)	Polish & Testing	Platform-wide UI/UX enhancements (Homepage redesign, shared layout); Complete UAT testing and final bug fixes.	Progress Report 5
Week 12 (Mar 31)	Deployment	Containerize application; Deploy live platform to Google Cloud Run; Finalize project documentation.	
Week 13 (Apr 7)	Defense	Final system testing of live deployment; Prepare final defense presentation slides; Submit final code.	Final Report & Defense

4.2 Team Responsibilities

The project was developed using a highly collaborative, iterative approach. While primary ownership was divided, both team members contributed to the full-stack implementation of several modules to accommodate the interactive challenge architecture:

- **Thilan Udahage Don:** Handled primary full-stack implementation for the CSRF, IDOR, and File Upload modules, alongside the frontend UI and sanitization logic for the Reflected XSS module. Led the platform-wide UI/UX redesign and progressive challenge architecture, which included refactoring the initial SQL Injection module into a multi-level challenge and updating the User Profile Tracker UI. Conceived and implemented the central 'Why It Worked' Explainer feature across all vulnerability modules.
- **Allwyn Mascarenhas (Team Lead):** Handled primary implementation for the Broken Authentication module. Led backend Spring Security configurations, database entity design (including the ChallengeProgress model), and developed the initial backend implementation for the SQL Injection and User Profile Progress Tracker modules. Additionally, he contributed crucial backend architecture to the Reflected XSS module by implementing its dedicated controller and Content Security Policy (CSP) configurations.

4.3 Project Management Chart

To ensure organized and concurrent development, the team utilized a **Kanban Board** methodology (via GitHub Projects) to track feature ownership, monitor milestone progress, and manage iterative refactoring. Below is a snapshot of the final project board at completion:

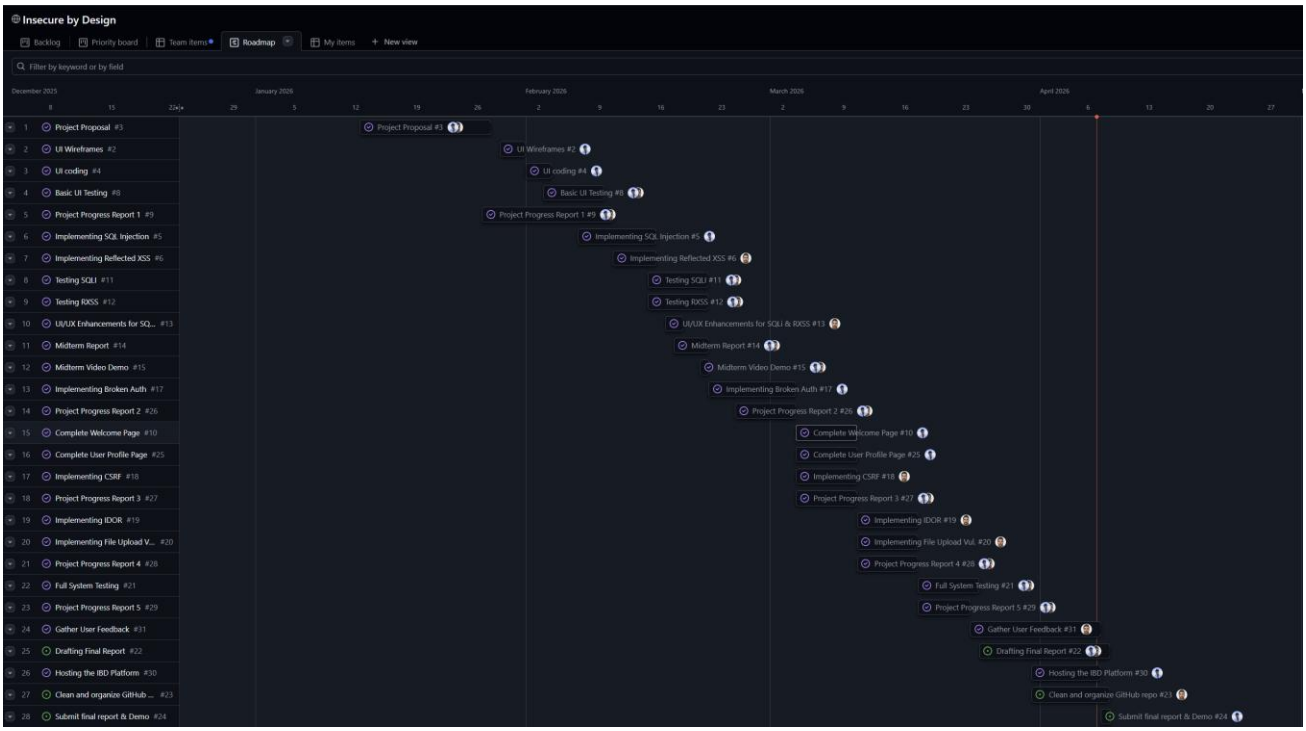


Figure 1 - Kanban board snapshot reflecting the agile project management workflow, task assignments, and completion status across the development lifecycle.

5. Implemented Features

5.1 Vulnerabilities and Exploits

The Insecure by Design platform the topmost critical OWASP vulnerability modules, each split into progressively harder challenge levels. Every level gives the student real payloads to try, explains why the vulnerability worked, and awards XP on success. The six vulnerabilities are described below.

5.1.1 SQL Injection

SQL Injection happens when user input is placed directly into a database query without being sanitised. An attacker can alter the query's logic to retrieve data they should not have access to, bypass login checks, or destroy records. It consistently ranks at or near the top of the OWASP Top 10.

The platform implements three challenge levels. Level 1 demonstrates classic string-based injection where a payload such as ' OR '1'='1 bypasses a login WHERE clause. Level 2 covers numeric injection (no surrounding quotes), and Level 3 introduces blind boolean injection, where the application only

reveals true or false but can still be exploited. The fix shown to students in all cases is a parameterised prepared statement.

The module uses SqlInjectionController for AJAX-based payload validation and SqlInjectionChallengeService for real query execution against the H2 database — meaning payloads run against actual SQL, not pattern matching. Level 2 and Level 3 are fully hidden from the HTML source until prerequisites are met, implemented using server-side Thymeleaf th:if conditions and per-level completion flags passed through the model.

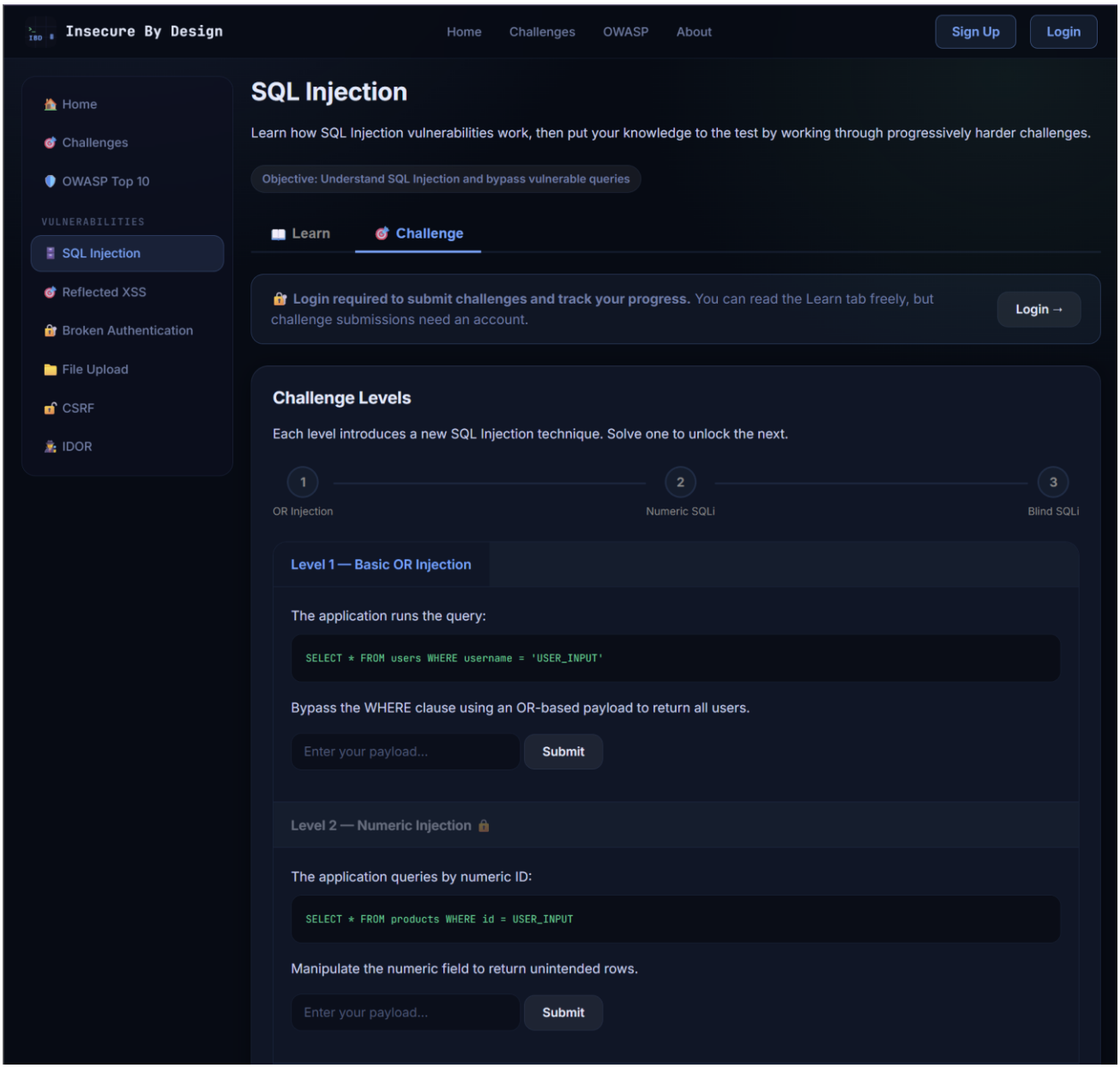


Figure 2 - The SQL Injection module interface, displaying the progressive challenge structure and the Level 1 input form.

```java
// ── Level 1 — OR Injection ──────────────────────────────────────────
// Vulnerable query: SELECT * FROM sqli_users WHERE username = 'USER_INPUT'
// Success condition: query returns more than 1 row (bypassed the WHERE clause)
public ChallengeResult checkLevel1(String payload) {
    if (payload == null || payload.isBlank()) {
        return ChallengeResult.fail( hint: "Please enter a payload.");
    }

    String query = "SELECT * FROM sqli_users WHERE username = '" + payload + "'";

    try (Connection conn = dataSource.getConnection();
         Statement stmt = conn.createStatement()) {

        // Prevent destructive statements
        String upper = payload.toUpperCase();
        if (upper.contains("DROP") || upper.contains("DELETE")
                || upper.contains("UPDATE") || upper.contains("INSERT")) {
            return ChallengeResult.fail(
                    hint: "Destructive statements are not allowed in this challenge.");
        }

        ResultSet rs = stmt.executeQuery(query);
        int rowCount = 0;
        while (rs.next()) rowCount++;

        if (rowCount > 1) {
            return ChallengeResult.pass(query);
        } else {
            return ChallengeResult.failWithQuery(query,
                    hint: "The query ran but only returned " + rowCount + " row(s). "
                            + "You need to make the WHERE clause return ALL users. "
                            + "Hint: use OR logic to make the condition always true.");
        }

    } catch (Exception e) {
        // SQL syntax error — give a nudge
        return ChallengeResult.failWithQuery(query,
                hint: "Your payload caused a SQL error. Check your quote characters and try again.");
    }
}
```

Figure 3 - Backend validation logic for the Level 1 SQL Injection challenge, demonstrating the execution of a user-supplied payload directly against the database to evaluate bypass success.

5.1.2 Reflected Cross-Site Scripting (XSS)

Reflected XSS occurs when a web page echoes user input back as raw HTML without encoding it first. A malicious script injected into the page runs in the victim's browser under the site's origin, giving an attacker access to cookies and session tokens. The root cause in this module is Thymeleaf's th:utext directive, which renders input as unescaped HTML.

Level 1 asks the student to inject a basic <script>alert(1)</script> tag. Level 2 strips script tags with a regex filter, requiring the student to bypass it using an event handler such as <img src=x

onerror=alert(1)>. Level 3 also strips angle brackets, so the student must break out of an HTML attribute value using a quote character to inject an event handler. The safe fix demonstrated is switching to th:text, which HTML-encodes all output automatically. The XssChallengeService implements the filter simulation for each level, and the ReflectedXssController handles AJAX validation and progress tracking. Each level unlocks only after the previous one is completed.

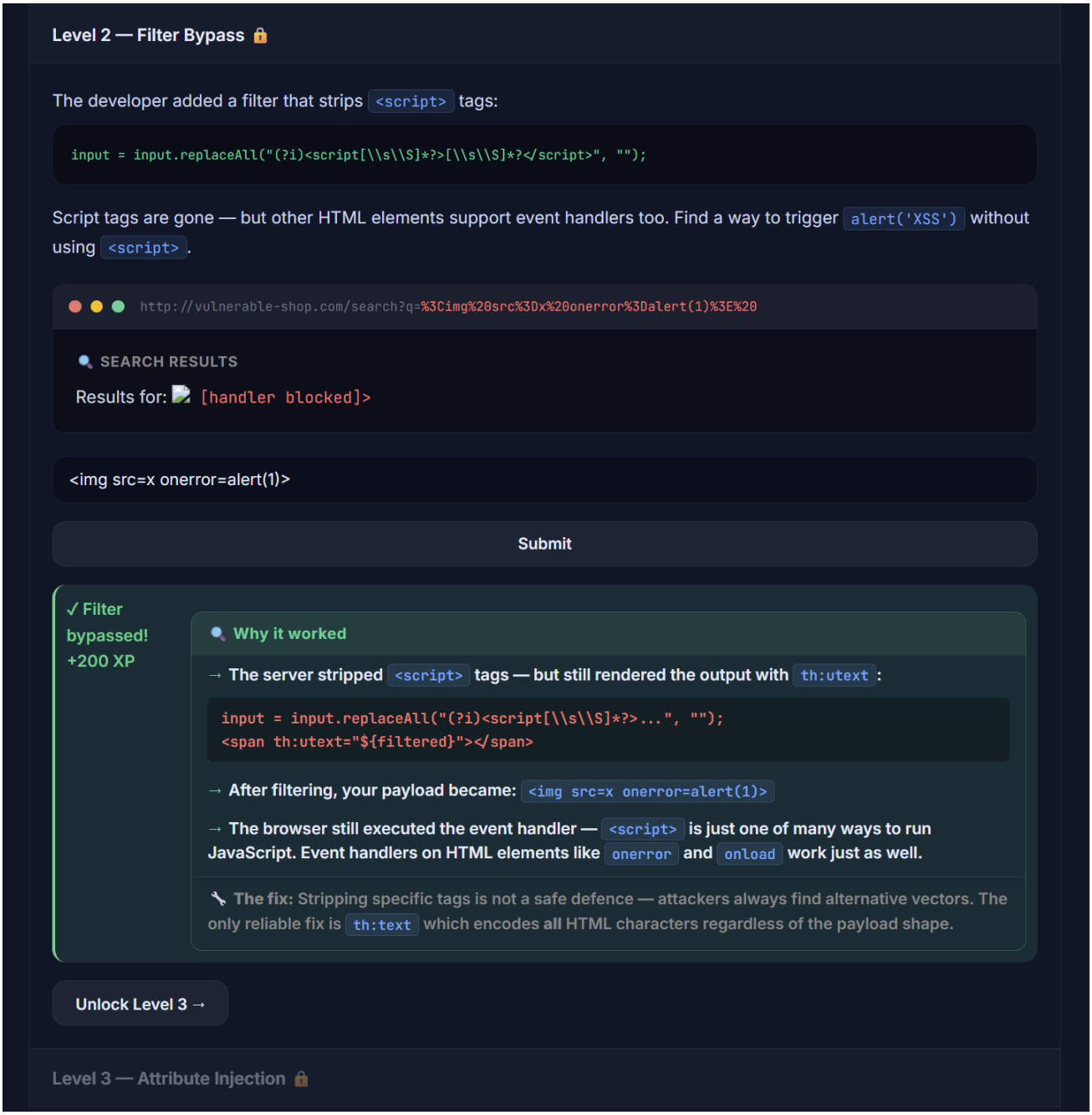


Figure 4 - Reflected XSS Level 2 interface showing a successful event-handler filter bypass and the dynamic reveal of the 'Why It Worked' explainer.

```java
/**
 * Level 2 — Filter Bypass
 * Server strips full <script>...</script> blocks (regex, case-insensitive).
 * Payload must NOT rely on script tags — must use an event-handler attribute.
 * Valid examples: <img src=x onerror=alert(1)>, <svg onload=alert(1)>
 */
public boolean checkLevel2(String payload) {
    if (payload == null || payload.isBlank()) return false;

    // Simulate the server-side filter
    String filtered = payload.replaceAll( regex: "(?i)<script[\\s\\S]*?>[\\s\\S]*?</script>",  replacement: "");

    String f = filtered.toLowerCase();

    // After filtering, must still contain an event handler attribute
    boolean hasEventHandler = f.matches( regex: "(?s).*\\bon(error|load|click|mouseover|focus|input|submit)\\s*=.*");

    // Reject if they just stuffed alert in something that got stripped anyway
    boolean hasPayload = f.contains("alert(") || f.contains("prompt(") || f.contains("confirm(")
            || f.contains("document.cookie") || f.contains("javascript:");

    return hasEventHandler && hasPayload;
}
```

Figure 5 - Backend validation logic in XssChallengeService simulating a regex-based script tag filter to evaluate the learner's ability to utilize alternative execution vectors.

5.1.3 Cross-Site Request Forgery (CSRF)

CSRF tricks a logged-in user's browser into sending a forged request to a site they are authenticated with. Because the browser automatically attaches the session cookie, the server cannot tell the request was not initiated by the real user. State-changing actions such as changing an email address are the primary targets.

Level 1 demonstrates a completely unprotected endpoint where a forged email value is accepted with no token check at all. Level 2 shows a developer attempt at protection using a hardcoded static token (csrf_token_123), which is visible in the page source and trivially bypassed. Students learn that a valid CSRF token must be cryptographically random, unique per session, and validated server-side on every state-changing request.

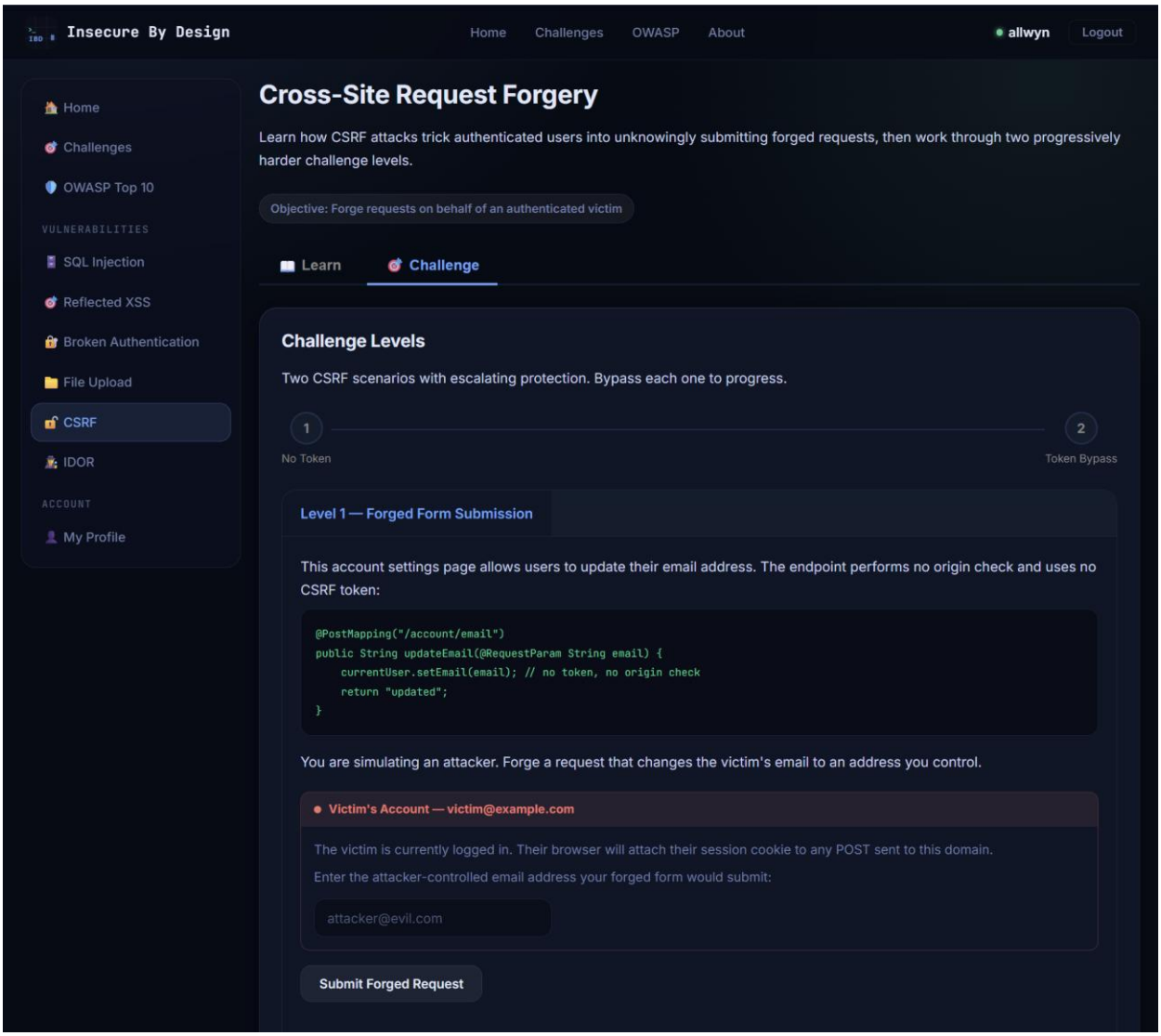


Figure 6 - he CSRF module interface, prompting the learner to submit a forged state-changing email update request on behalf of an authenticated victim.

```java
public class CsrfChallengeService {

    // Level 1 — Forged Form Submission
    // Simulates a state-changing action (email change) that has no CSRF token.
    // Student must submit a forged action targeting the victim's account.
    // We accept any non-blank email that looks like a forged attacker-controlled address.
    public boolean checkLevel1(String forgedEmail) {
        if (forgedEmail == null || forgedEmail.isBlank()) return false;
        String e = forgedEmail.trim().toLowerCase();
        // Accept any email that is not the victim's own address — simulating an attacker-controlled value
        return e.contains("@") && !e.equals("victim@example.com");
    }

    // Level 2 — Predictable / Missing Token Bypass
    // The app now adds a CSRF token, but it's static and hardcoded — trivially guessable.
    // Student must submit the correct static token to simulate bypassing the weak protection.
    private static final String WEAK_TOKEN = "csrf_token_123";  1 usage

    public boolean checkLevel2(String token) {
        if (token == null) return false;
        return token.trim().equals(WEAK_TOKEN);
    }
}
```

Figure 7 - Backend implementation of the CSRF challenges, depicting an intentionally unprotected state-changing endpoint alongside a trivially guessable static token validation check.

5.1.4 Insecure Direct Object Reference (IDOR)

IDOR occurs when an application uses a user-controllable value such as a numeric ID to look up a resource without checking whether the requesting user is actually authorised to access it. Because database IDs are often sequential, an attacker can simply guess or increment a value to access other users' data.

Level 1 asks the student to access another user's profile by supplying a different ID in the request. Level 2 escalates this by having the student access the admin profile by predicting that the first registered user would have ID 1. The fix shown is to always verify that the requested object belongs to the authenticated user before returning it, and to use non-sequential UUIDs for sensitive resources.

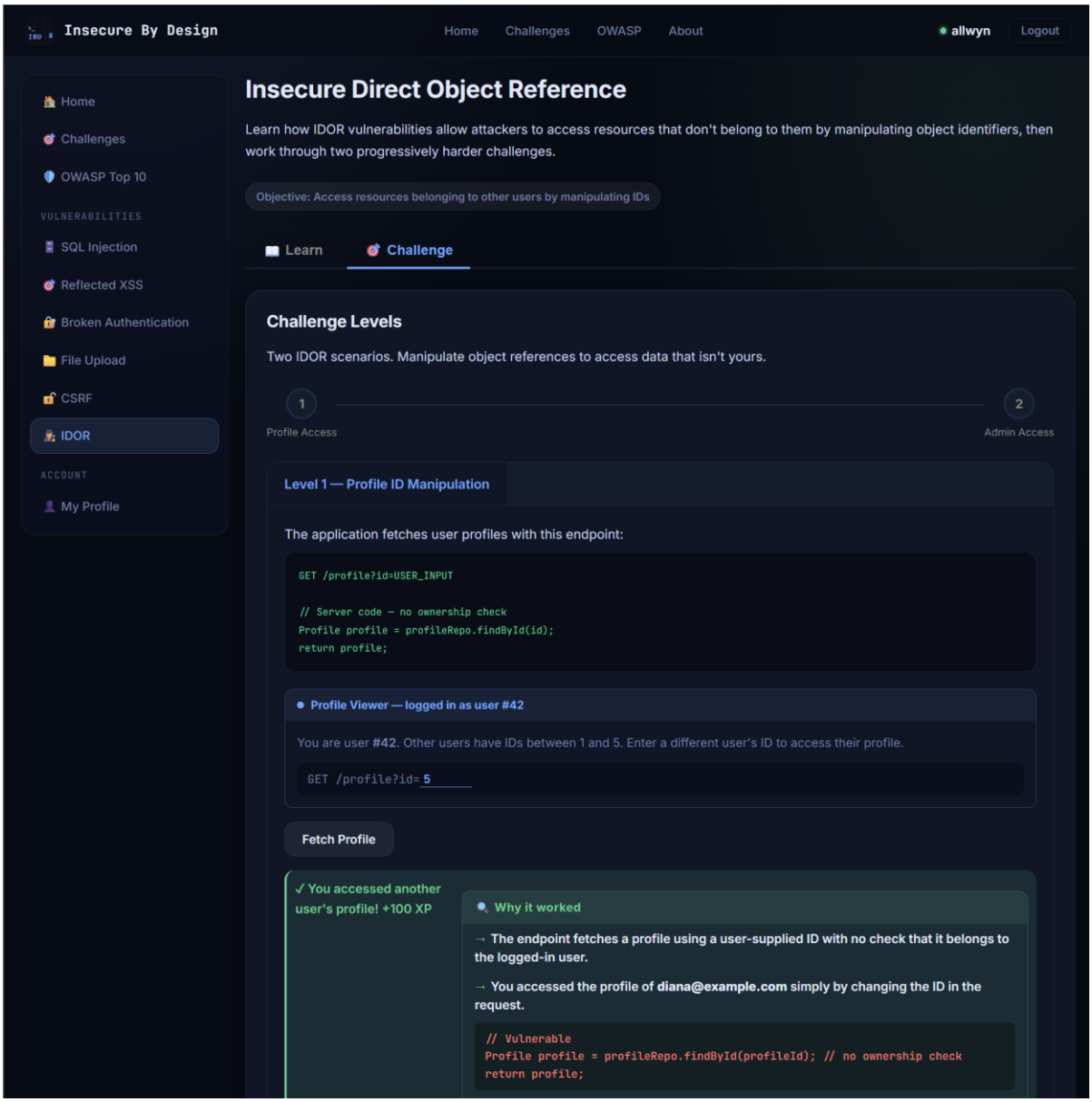


Figure 8 - The IDOR challenge interface showing the successful manipulation of a predictable profile ID parameter to access unauthorized user data.

```java
public class IdorChallengeService {

    // Simulated user profile store — the "database" of profiles accessible by ID
    // The logged-in user is user ID 42. IDs 1-5 belong to other users including the admin (ID 1).
    private static final Map<Integer, String> PROFILES = Map.of(  2 usages
            k1: 1,   v1: "admin@example.com",
            k2: 2,   v2: "alice@example.com",
            k3: 3,   v3: "bob@example.com",
            k4: 4,   v4: "charlie@example.com",
            k5: 5,   v5: "diana@example.com",
            k6: 42,  v6: "you@example.com"
    );

    // Level 1 — Profile ID Manipulation
    // The student is logged in as user 42. They must access another user's profile
    // by changing the ID parameter to any value other than their own (42).
    public boolean checkLevel1(int profileId) {
        return profileId != 42 && PROFILES.containsKey(profileId);
    }

    // Returns the profile email for display — used in the success response
    public String getProfileEmail(int profileId) {  1 usage
        return PROFILES.getOrDefault(profileId,  defaultValue: "unknown");
    }

    // Level 2 — Admin Profile Access
    // The student must specifically reach the admin account (ID 1) by predicting
    // that the first registered user — the admin — has ID 1.
    public boolean checkLevel2(int profileId) {
        return profileId == 1;
    }
}
```

Figure 9 - Backend logic for the IDOR module simulating a user profile database query that fundamentally lacks resource ownership authorization checks.

5.1.5 Broken Authentication

Broken Authentication covers weaknesses in how applications verify who a user is. Common examples include default credentials that are never changed, weak or predictable session tokens, and credential stuffing, where attackers reuse leaked username/password pairs from data breaches against other sites.

Level 1 challenges the student to log in using default credentials that the application ships with. Level 2 requires identifying and submitting a predictable session token based on observable patterns. Level 3 simulates a credential stuffing attack using a sample breach list. Completing all three levels earns the Auth Cracker badge.

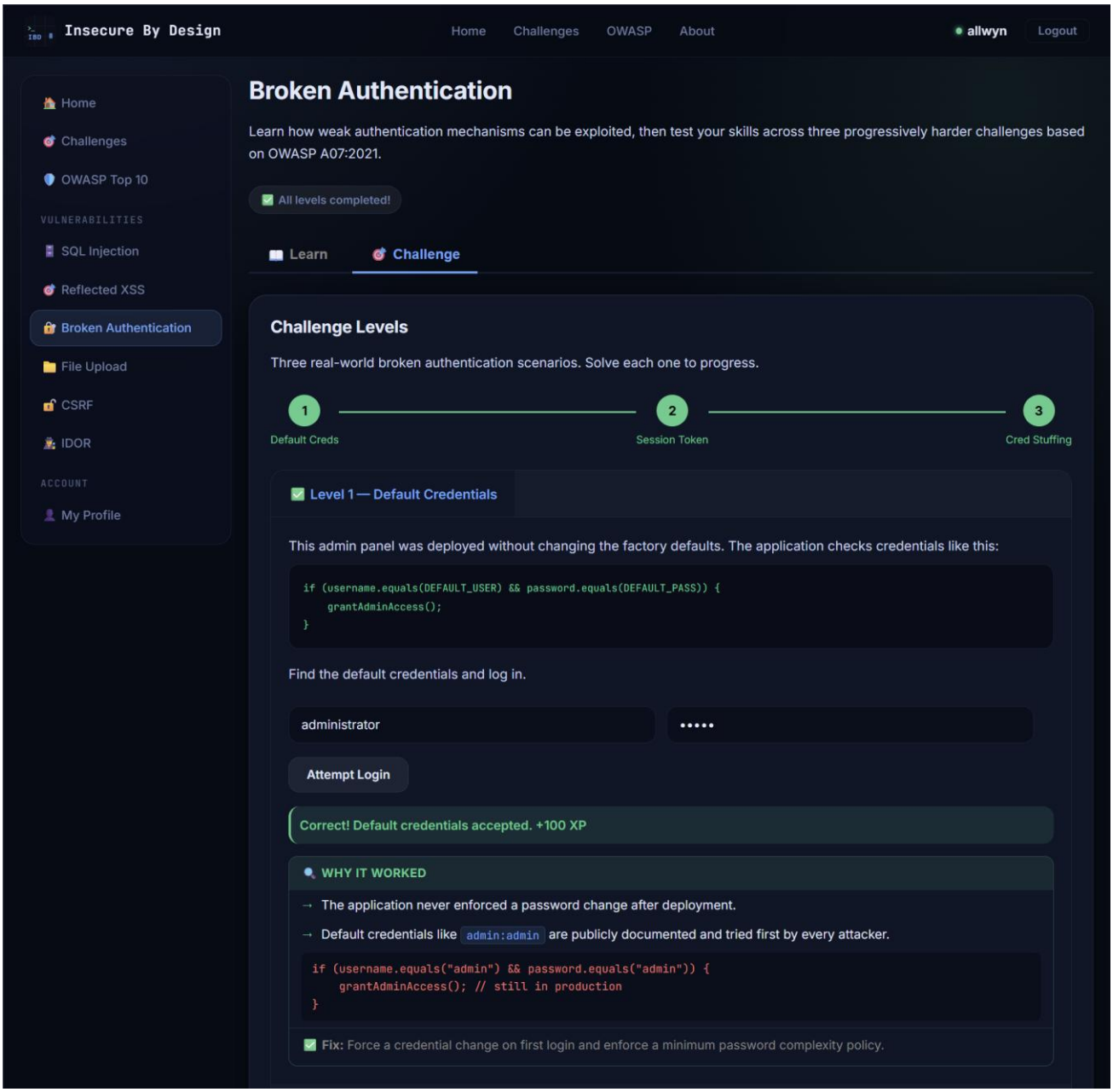


Figure 10 - The Broken Authentication module interface demonstrating the successful exploitation of hardcoded default administrative credentials.

```java
public boolean checkLevel2(String token) {
    if (token == null) return false;
    String t = token.trim();

    // Accept any of the known weak tokens
    if (WEAK_TOKENS.contains(t)) return true;

    // Also accept if it decodes cleanly to a simple word (base64 check)
    try {
        String decoded = new String(Base64.getDecoder().decode(t)).toLowerCase();
        return decoded.equals("admin") || decoded.equals("user1")
                || decoded.equals("user-1") || decoded.equals("session1");
    } catch (IllegalArgumentException e) {
        // Not valid base64 — that's fine, still check other cases
    }

    // Accept sequential numeric-style tokens: session-1, session-2, user-1 etc.
    return t.matches( regex: "(user|session|token|id)-?\\d+");
}
```

Figure 11 - Backend validation logic evaluating predictable session tokens, including Base64 decoding attempts and sequential ID pattern matching.

5.1.6 Insecure File Upload

Insecure file upload vulnerabilities arise when a server accepts uploaded files without properly validating their type or content. If an attacker can upload an executable file such as an HTML page with embedded JavaScript and the server serves it back to the browser, the script runs under the site's origin, effectively achieving stored XSS.

Level 1 has no validation at all — any file is accepted and served back, and the student can visit the resulting URL to trigger script execution. Level 2 adds a MIME-type check that trusts the browser-supplied Content-Type header, which can be spoofed using a tool such as Burp Suite. Level 3 implements an extension blacklist that can be bypassed using a double extension such as payload.png.html. The secure fix is to serve all uploaded files with a Content-Disposition: attachment header so the browser downloads rather than renders them.

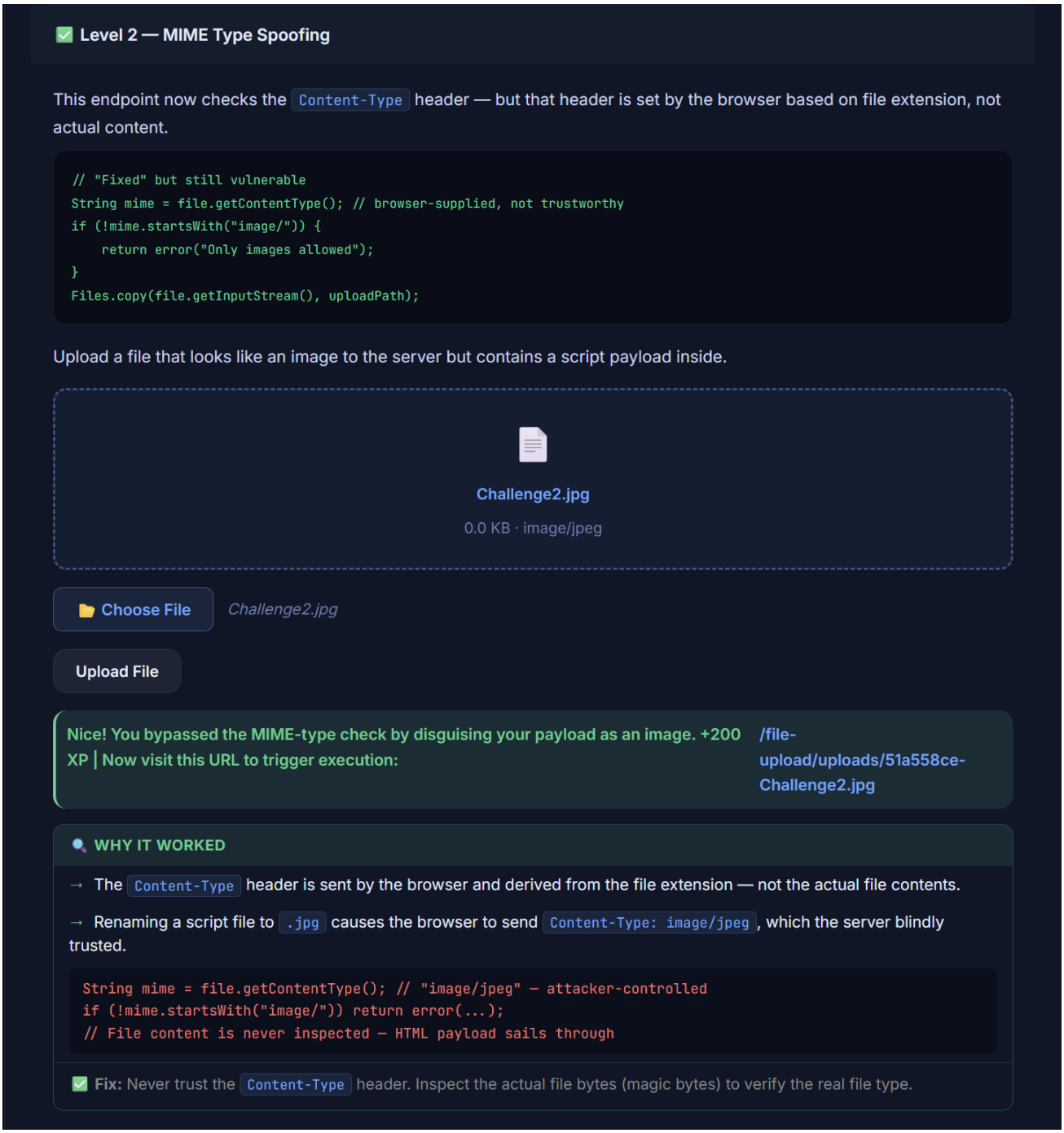


Figure 12 - Insecure File Upload module interface showing a successful MIME type spoofing attack and the generation of a direct execution link.

```java
// ── Level 2 ──────────────────────────────────────────────────────────
// App checks Content-Type header only (client-supplied, easily spoofed).
// Student uploads a file with a .jpg extension but HTML/script content inside.
// Simulates bypassing a MIME-type-only check.
public boolean checkLevel2(MultipartFile file) {
    if (file == null || file.isEmpty()) return false;

    String filename = file.getOriginalFilename();
    if (filename == null) return false;

    String lower = filename.toLowerCase();

    // Must look like an image by extension (the "bypass")
    boolean looksLikeImage = lower.endsWith(".jpg") || lower.endsWith(".jpeg")
            || lower.endsWith(".png") || lower.endsWith(".gif");

    if (!looksLikeImage) return false;

    // But must contain a script/HTML payload inside (the actual attack)
    try {
        String content = new String(file.getBytes()).toLowerCase();
        return content.contains("<script") || content.contains("<html")
                || content.contains("onerror=") || content.contains("onload=");
    } catch (Exception e) {
        return false;
    }
}
```

Figure 13 - File Upload — Level 2 checking extension looks like image while content contains script

5.2 User Features

5.2.1 Why It Worked' Explainer

The 'Why It Worked' Explainer is the platform's central novel feature. Each challenge module includes a hidden explanation section that is revealed only after a learner successfully completes a challenge level. The reveal is triggered by the AJAX success response using JavaScript, ensuring the explanation is never accessible to learners who have not yet attempted the challenge. The Explainer content for each module covers the attack mechanism, the vulnerable code pattern being exploited, and the real-world impact of the vulnerability. Unlike platforms such as TryHackMe, which lack post-challenge educational sections, this feature directly addresses the primary educational gap identified in existing cybersecurity training tools. Because it explains the exact vulnerable code pattern just exploited, it serves as the core novelty and academic contribution of the platform.

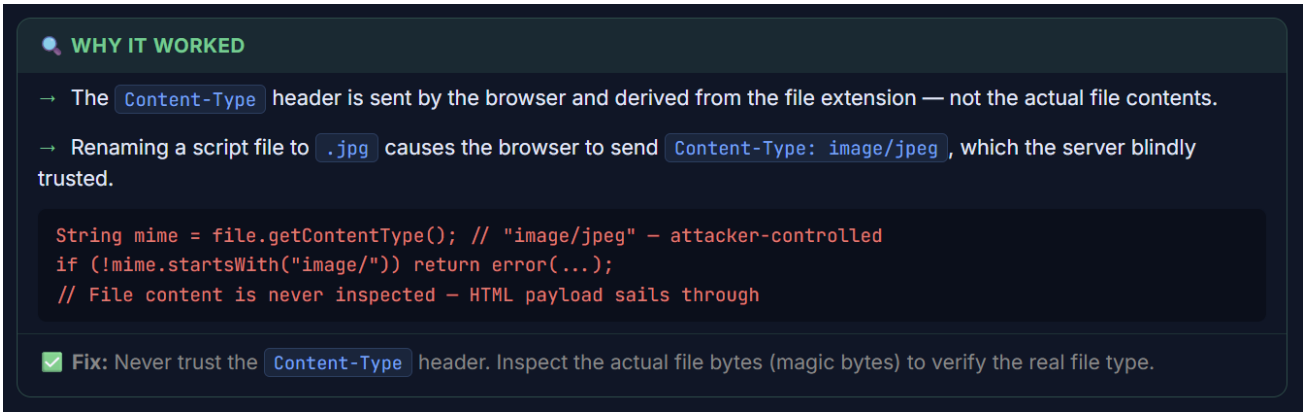
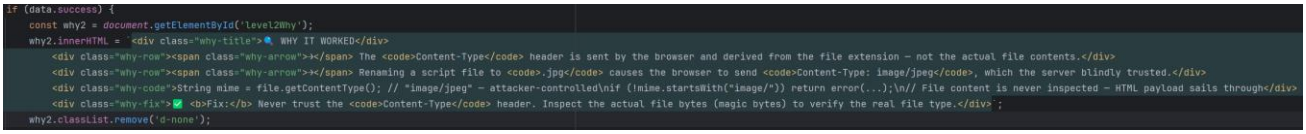


Figure 14 - The 'Why It Worked' Explainer dynamically revealed after a successful exploit, detailing the vulnerable code pattern and industry-standard remediation strategy.



```
if (data.success) {
    const why2 = document.getElementById('level2Why');
    why2.innerHTML = `<div class="why-title">🔍 WHY IT WORKED</div>
        <div class="why-row"><span class="why-arrow">→</span> The <code>Content-Type</code> header is sent by the browser and derived from the file extension — not the actual file contents.</div>
        <div class="why-row"><span class="why-arrow">→</span> Renaming a script file to <code>.jpg</code> causes the browser to send <code>Content-Type: image/jpeg</code>, which the server blindly trusted.</div>
        <div class="why-code">String mime = file.getContentType(); // "image/jpeg" — attacker-controlled\nif (!mime.startsWith("image/")) return error(...);\n// File content is never inspected — HTML payload sails through</div>
        <div class="why-fix">✅ <b>Fix:</b> Never trust the <code>Content-Type</code> header. Inspect the actual file bytes (magic bytes) to verify the real file type.</div>`;
    why2.classList.remove('d-none');
```

Figure 15 - Client-side JavaScript logic responsible for dynamically rendering the contextual remediation advice and code snippets upon a successful AJAX validation response.

5.2.2 User Profile Progress Tracker

The user profile progress tracker provides learners with a visual dashboard of their challenge completion status. The ProfileController fetches per-challenge completion data from the ChallengeProgress entity and passes it to the profile.html Thymeleaf template, where Bootstrap badges visually indicate completed and incomplete levels for each module. This feature enables self-directed progress monitoring and provides learners with a clear view of remaining challenges.

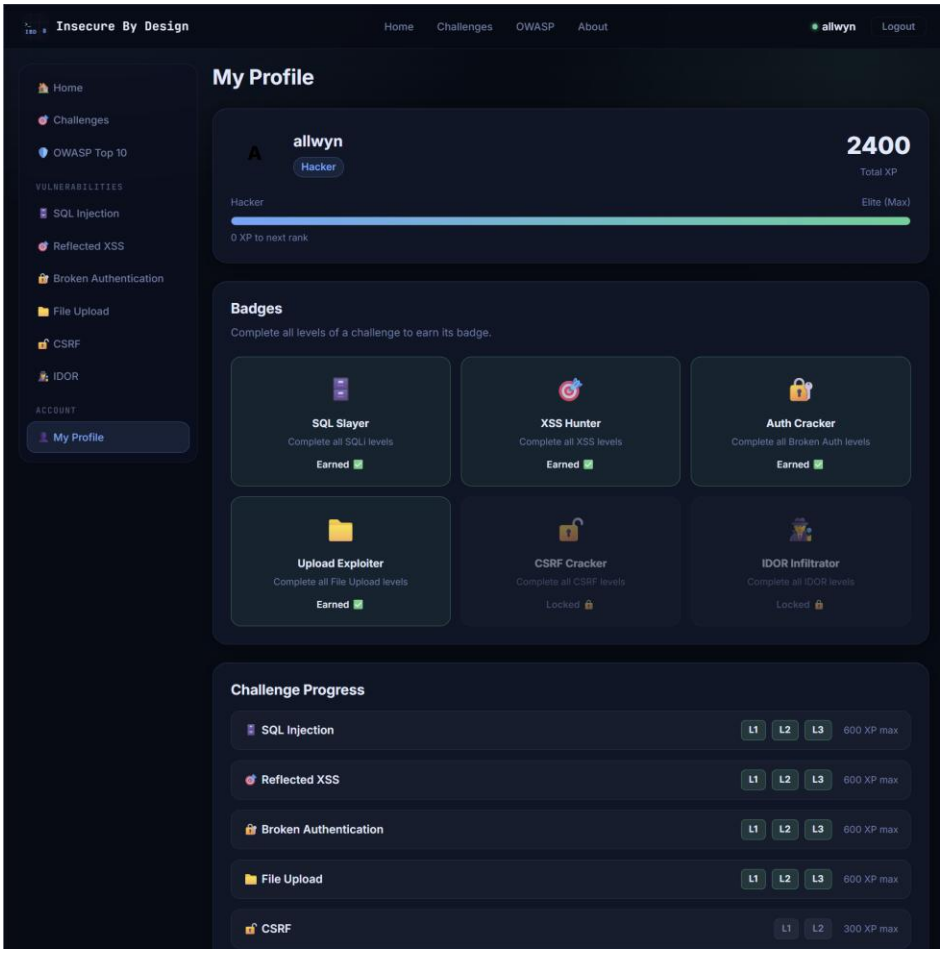


Figure 16 - The User Profile dashboard utilizing Bootstrap badges and progress bars to display real-time chronological progression and earned module achievements.

5.2.3 Homepage and Platform UI

The homepage was redesigned with a hero section providing a platform overview, Bootstrap challenge cards linking to each module, and a features section describing the platform's educational approach. The shared layout template includes a consistent navbar and footer across all pages. An OWASP reference page was added linking each implemented module to its corresponding OWASP Top 10 category, providing learners with external context for each vulnerability type.

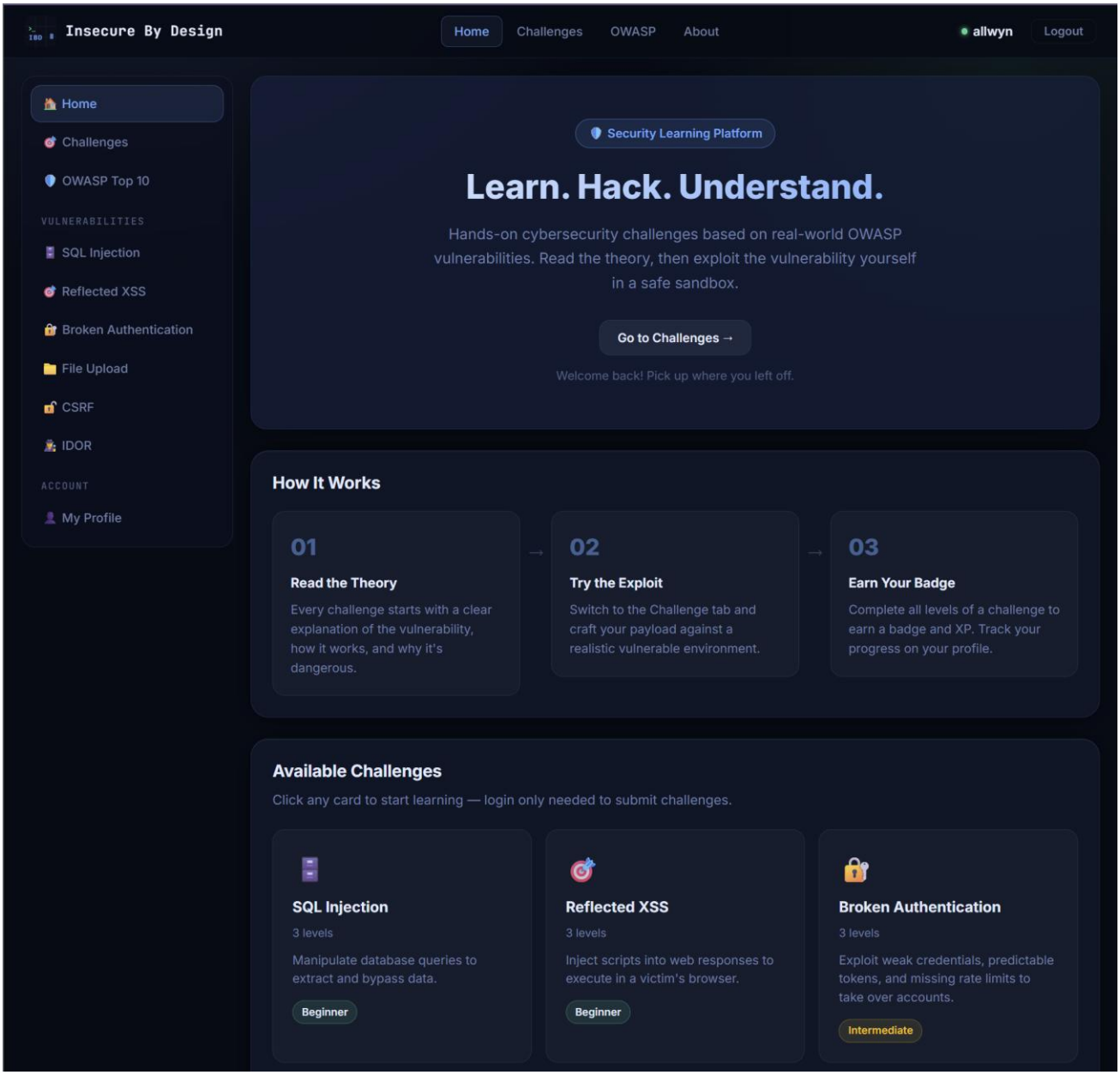


Figure 17 - The redesigned platform homepage featuring a modernized hero section, platform overview, and modular challenge routing.

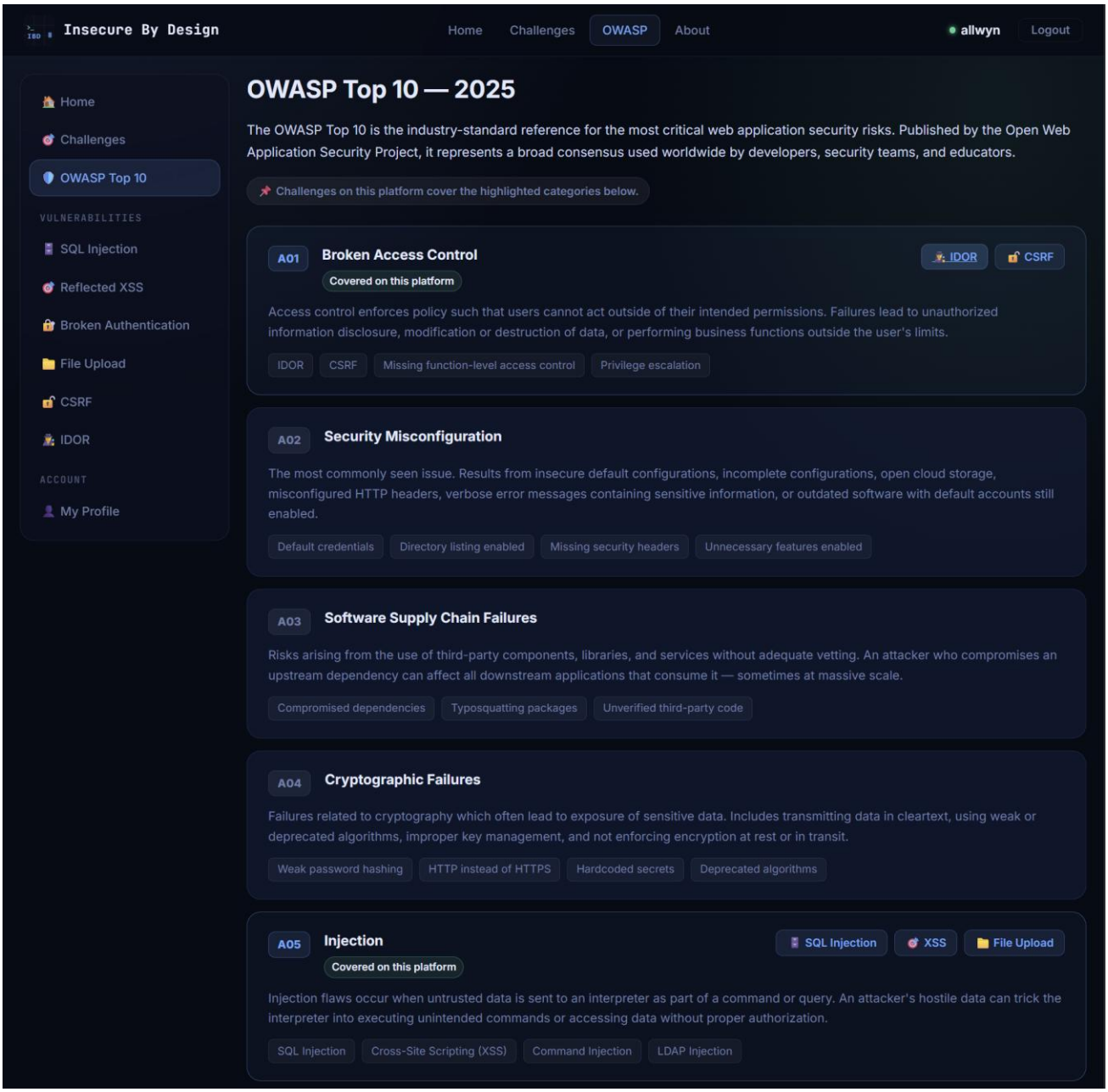


Figure 18 - The dedicated OWASP Top 10 reference architecture, explicitly mapping the platform's interactive challenges to their corresponding real-world risk categories.

6. Evaluation

To ensure the platform met both its technical security requirements and its primary educational objectives, the project was evaluated using a two-pronged approach: Functional User Acceptance Testing (UAT) and Qualitative User Testing via surveys.

6.1 Functional Evaluation: User Acceptance Testing (UAT)

Before conducting user trials, the platform underwent rigorous User Acceptance Testing. Because the system is an intentionally vulnerable application, standard unit testing was insufficient; the platform had to be evaluated on its ability to successfully allow exploits while maintaining overall application stability and state progression. The UAT protocol included validation of the "happy path" (successful payload execution) and constraint testing (ensuring progressive locking mechanisms functioned correctly).

6.1.1 General System Checks

#	What to do	Expected result	Notes / what to put	Pass / Fail
G-1	Visit /signup and register a new user	Redirects to /login?registered	Username 3–30 chars, password ≥ 6 chars (e.g. testuser / pass123)	Pass
G-2	Try to register with the same username again	Error: "Username 'x' is already taken"	Use the same username from G-1	Pass
G-3	Try to register with short username/password	Error: "Username must be 3–30 characters…"	Username = ab, password = 123	Pass
G-4	Log in with valid credentials	Redirected to home, session active	Use account from G-1	Pass
G-5	Visit /profile while logged in	Profile page loads, shows username + XP rank bar	No extra input — just navigate	Pass
G-6	Visit /challenges while logged in	Challenge hub shows progress state for each module	No extra input	Pass
G-7	Visit /profile while NOT logged in	Redirected to login page	Log out first, then navigate to /profile	Pass

6.1.2 Broken auth

#	What to do	Expected result	Notes / what to put	Pass / Fail
BA-1	Submit Level 1 with wrong credentials	Failure message about default credentials	Username `wrong`, password `wrong`	Pass
BA-2	Submit Level 1 with default credentials	Success + 100 XP awarded	Try common defaults like `admin / admin`	Pass
BA-3	Submit Level 2 with a random token	Failure — token not accepted	Token = `randomstring123`	Pass
BA-4	Submit Level 2 with a predictable/weak session token	Success + 200 XP awarded	Token = `user-1` or similar sequential value	Pass
BA-5	Submit Level 3 with a wrong email/password pair	Failure — no match in breach list	Email `nobody@example.com`, password `abc`	Pass
BA-6	Submit Level 3 with a valid breach-list pair	Success + 300 XP; badge if L1 & L2 done first	Use a credential stuffing pair from the hint	Pass
BA-7	Complete L1+L2 then L3 in order	"Auth Cracker" badge awarded	Complete all three levels sequentially	Pass

6.1.3 SQL INJ

#	What to do	Expected result	Notes / what to put	Pass / Fail
SQ-1	Submit Level 1 with normal username	Failure + built query shown	Payload = `alice`	Pass
SQ-2	Submit Level 1 with classic SQLi payload	Success + 100 XP + "Why it worked" explanation	Payload = `' OR '1'='1`	Pass
SQ-3	Submit Level 2 with a normal integer ID	Failure	Payload = `1`	Pass
SQ-4	Submit Level 2 with numeric injection	Success + 200 XP + "Why it	Payload = `1 OR 1=1`	Pass

		worked" explanation		
SQ-5	Submit Level 3 with a false condition	Failure (boolean blind — no data returned)	Payload = `1 AND 1=2`	Pass
SQ-6	Submit Level 3 with a true blind condition	Success + 300 XP; "SQL Slayer" badge if L1+L2 done	Payload = `1 AND 1=1`	Pass

6.1.4 XSS Reflected

#	What to do	Expected result	Notes / what to put	Pass / Fail
XS-1	Submit Level 1 with plain text	Failure — hint to use a script tag	Payload = `hello`	Pass
XS-2	Submit Level 1 with a script-tag payload	Success + 100 XP + "Why it worked" explanation	Payload = `<script>alert(1)</script>`	Pass
XS-3	Submit Level 2 with a <script> tag	Failure — script tag was stripped, hint given	Payload = `<script>alert(1)</script>`	Pass
XS-4	Submit Level 2 with an event-handler bypass	Success + 200 XP + "Why it worked" explanation	Payload = `<img src=x onerror=alert(1)>`	Pass
XS-5	Submit Level 3 with angle brackets	Failure — angle brackets stripped, hint given	Payload = `<script>alert(1)</script>`	Pass
XS-6	Submit Level 3 with attribute-injection payload	Success + 300 XP; "Script Kiddie" badge if L1+L2 done	Payload = `" onmouseover="alert(1)`	Pass

6.1.5 Insecure file upload

#	What to do	Expected result	Notes / what to put	Pass / Fail
FU-1	Upload a plain text file to Level 1	Failure — hint about browser-executable file types	Upload any `.txt` file	Pass

FU-2	Upload an HTML file with a script to Level 1	Success + 100 XP + link to visit the uploaded file	Create `payload.html` with `<script>alert(1)</script>`	Pass
FU-3	Visit the served file URL from FU-2	Browser renders the HTML and runs the script	Click the link returned in the success message	Pass
FU-4	Upload a real image file to Level 2	Failure — hint about MIME type check	Upload a normal `.jpg` image	Pass
FU-5	Upload HTML disguised as an image (MIME bypass) to Level 2	Success + 200 XP	Set Content-Type to `image/png` but file is HTML (use Burp/curl)	Pass
FU-6	Upload a file with a blacklisted extension to Level 3	Failure — blacklist blocks it	Upload `payload.html` directly	Pass
FU-7	Upload a file with a double extension to Level 3	Success + 300 XP; "Upload Exploiter" badge if L1+L2 done	Rename file to `payload.png.html` or `payload.html.png`	Pass

6.1.6 IDOR

#	What to do	Expected result	Notes / what to put	Pass / Fail
ID-1	Submit Level 1 with your own profile ID	Failure — must be another user's ID	Try ID = `999` (non-existent)	Pass
ID-2	Submit Level 1 with another user's valid ID	Success + 100 XP + victim's email shown in explanation	Try IDs like `2`, `3`, `4`	Pass
ID-3	Submit Level 2 with a non-admin ID	Failure — hint about first-registered user ID	Try ID = `5`	Pass
ID-4	Submit Level 2 with admin ID (ID = 1)	Success + 200 XP; "Reference Manipulator" badge if L1 done	Profile ID = `1`	Pass

6.1.7 CSRF

#	What to do	Expected result	Notes / what to put	Pass / Fail
CS-1	Submit Level 1 with a random email value	Failure — hint about forging the victim's email	Forged email = `nobody@test.com`	Pass

CS-2	Submit Level 1 with the correct forged email	Success + 100 XP + "Why it worked" explanation	Forged email = `attacker@evil.com` (as shown in page example)	Pass
CS-3	Submit Level 2 with a random CSRF token	Failure — wrong token	Token = `anything123`	Pass
CS-4	Find the hardcoded token in page source and submit it	Success + 200 XP; "Forgery Expert" badge if L1 done	View page source → find `csrf_token_123` (or similar) → submit it	Pass

6.1.8 XP and Progression

#	What to do	Expected result	Notes / what to put	Pass / Fail
XP-1	Complete a challenge level and check `/profile`	XP total increases, rank bar updates	Complete any level for the first time	Pass
XP-2	Repeat a completed challenge level	Success message shown but XP is not added again	Submit a winning payload on an already-solved level	Pass
XP-3	Reach 300 XP (Apprentice threshold)	Rank changes from Beginner to Apprentice on profile	Accumulate XP across challenges	Pass

6.2 Educational Evaluation: User Testing Methodology

Following technical validation, the platform's pedagogical effectiveness was evaluated through user testing with ten fellow computer science students. Participants were asked to work through at least two challenge modules independently without prior instruction, then complete a structured feedback survey covering ten questions across four areas: usability, learning experience, challenge modules, and overall satisfaction.

The ten feedback questions were:

1. How easy was it to understand what you had to do in each challenge? (1–5 scale)
2. How clear was the feedback you received after submitting a payload? (1–5 scale)

3. Did the progressive level structure help you build understanding incrementally? (Multiple choice)
4. How helpful was the 'Why It Worked' explanation shown after completing a challenge level? (1–5 scale)
5. After completing a challenge, did you feel you understood why the vulnerability existed — not just how to exploit it? (Multiple choice)
6. Which challenge module did you find most educational? (Multiple choice)
7. Did you feel stuck at any point without enough guidance to proceed? (Multiple choice)
8. Compared to reading about web vulnerabilities in textbooks or articles, how would you rate this hands-on approach? (Multiple choice)
9. Overall, how would you rate the Insecure by Design platform as a web security learning tool? (1–5 scale)
10. Is there anything you would change or add to improve the platform? (Open text, optional)

6.3 Educational Evaluation: Survey Results

The survey results indicate an overwhelmingly positive reception among the ten participating CS students, particularly regarding the platform's pedagogical effectiveness. Every respondent (100%) agreed that the progressive level structure significantly aided their incremental understanding, and 80% of users rated the hands-on approach as "much more effective" than traditional textbooks or articles.

A key driver of this success was the "Why It Worked" explainer, which served as the project's primary novelty. 90% of participants rated this feature a perfect 5/5, noting that it successfully bridged the gap between mere execution and actual theoretical comprehension. The platform's educational impact is further underscored by the fact that 80% of users felt they "clearly understood" the underlying vulnerabilities after completion, with SQL Injection and Broken Authentication cited as the most

impactful modules. Overall satisfaction was high, with 80% of users giving the tool a perfect 5/5 rating.

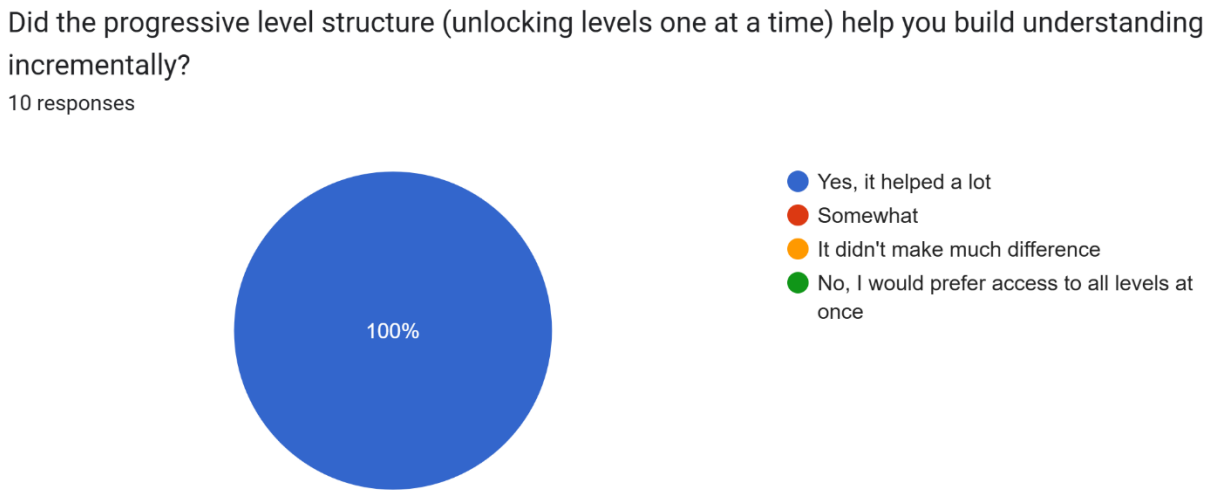


Figure 19 - User survey results indicating 100% agreement among participants that the progressive challenge structure significantly aided incremental understanding.

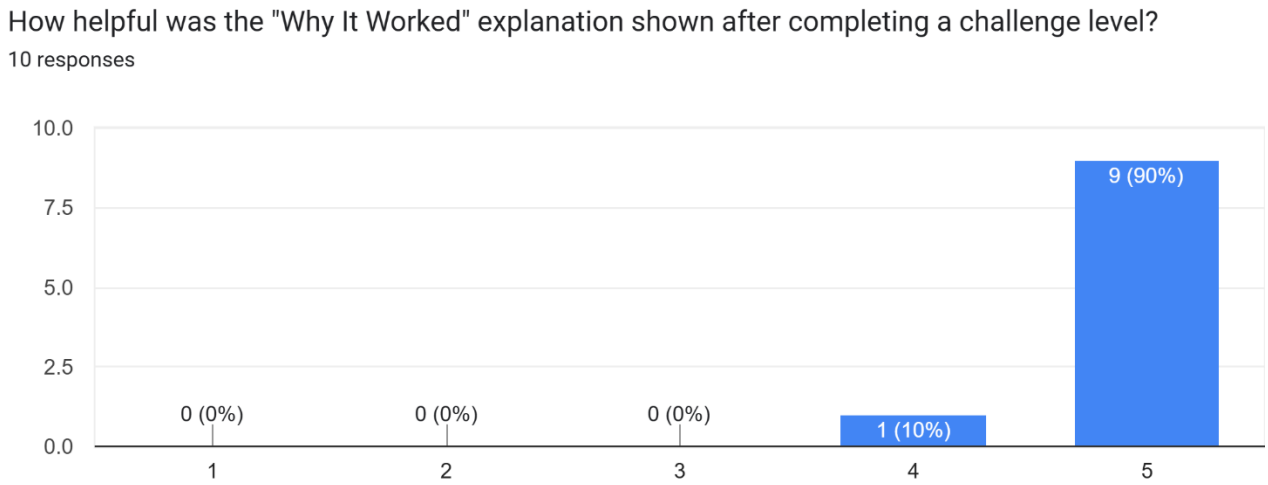


Figure 20 - User survey results demonstrating a 90% perfect satisfaction rating for the educational value of the post-challenge 'Why It Worked' explainer.

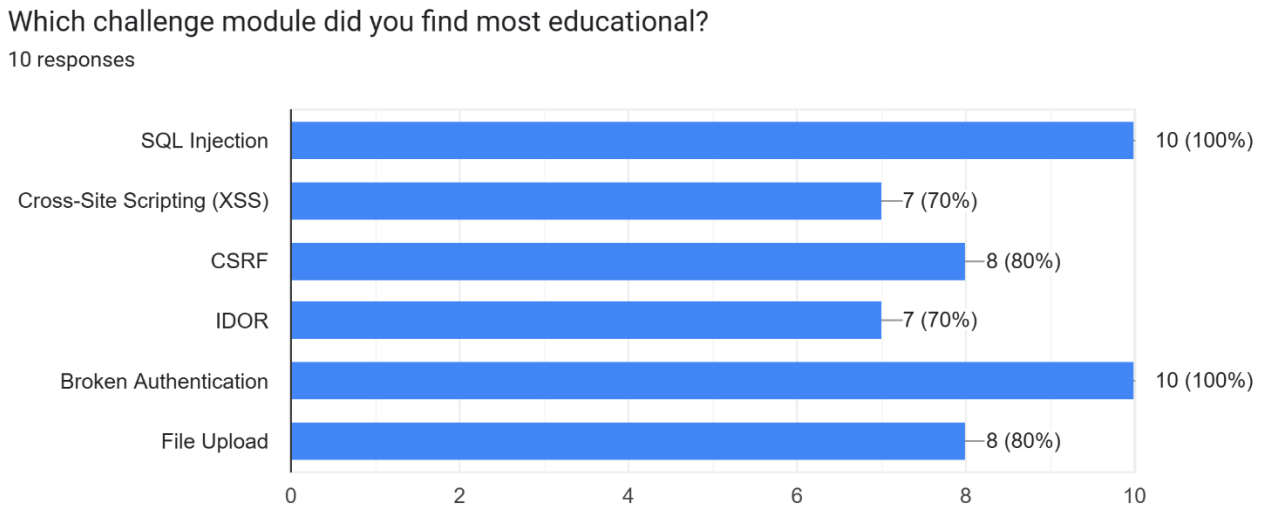


Figure 21 - User survey results identifying Broken Authentication and SQL Injection as the most impactful and educational challenge modules.

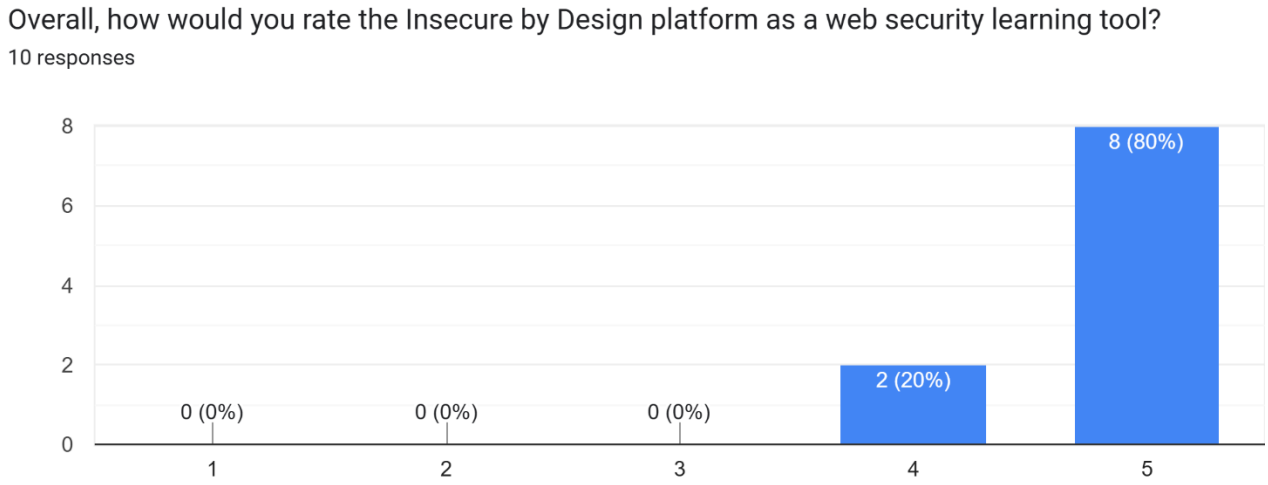


Figure 22 - User survey results reflecting a high overall satisfaction rating (80% perfect 5/5) for the pedagogical effectiveness of the Insecure by Design platform.

6.4 Design and Implementation Changes Driven by Evaluation

Based on the results of both the functional UAT and the qualitative user surveys, several critical architectural and design changes were finalized:

- **Security Header Adjustments (from UAT):** During functional testing of the Reflected XSS and File Upload modules, UAT revealed that modern browser security features (like strict MIME-type sniffing) were actively blocking intentional payloads from executing. To fix this, backend changes were implemented, including overriding the Content-Security-Policy to Report-Only and explicitly disabling X-Content-Type-Options, ensuring the exploits would execute as intended for the learner.
- **Refining the 'Why It Worked' Flow (from User Testing):** Early qualitative feedback indicated that learners sometimes missed the remediation advice if it blended in with the code snippets. Based on this, the UI was updated to ensure the "Fix" section of the Explainer was visually distinct and placed directly below the vulnerable code block, which contributed to the 90% satisfaction rate for this feature.
- **Curriculum Expansion (from User Testing):** Open-text feedback from the survey suggested a strong desire for additional challenges to be added to the curriculum. While outside the scope of the current term, this feedback drove the final architectural decision to make the ChallengeProgress entity highly modular, allowing future developers to easily add new vulnerability modules without rewriting the core tracking logic.

7. Reflections and Discussion

The most significant technical challenge of the project was the mid-term pivot from a demo-based to a challenge-based architecture. This required a complete redesign of the SQL Injection module, rethinking the data model for progress tracking, and replanning the implementation schedule. While disruptive, this change ultimately produced a substantially stronger product and reinforced the value of remaining responsive to instructor feedback during the development process.

Implementing server-side level-hiding using Thymeleaf's th:if conditions — rather than relying on client-side disabling — was a technically important decision. It ensured that challenge content for locked levels was never present in the HTML source, preventing learners from bypassing the progressive structure through browser developer tools.

The 'Why It Worked' Explainer presented an interesting design challenge: the explanation needed to be accessible only after successful exploitation, but the content needed to be present in the template

for the JavaScript reveal to work. This was resolved by rendering the Explainer sections with CSS display:none and toggling visibility via the AJAX success callback, ensuring the content is technically present but not readable without completing the challenge.

Working as a two-person team required clear division of responsibilities and consistent communication. Weekly Friday and Saturday meetings were effective in maintaining alignment, though tighter integration testing earlier in the term would have caught some of the profile tracker and broken authentication integration issues that were discovered later. The use of a shared GitHub repository with individual commits per team member provided clear accountability throughout the project.

The most satisfying outcome of this project was seeing the Explainer feature resonate with user testers. The immediate positive feedback on that feature validated the core educational design decision and affirmed the value of moving beyond challenge-only training platforms.

8. AI Use Section

8.1 Table of AI Tools and Specific Use

AI Tool Name	Version, Account Type	Specific Feature Used	Value Addition
Microsoft Copilot	Web interface, free	Generated boilerplate code snippets for Spring Boot controllers and Thymeleaf templates and provided explanations for Spring Security configurations.	Provided rapid code examples and technical explanations. All generated snippets were manually reviewed, modified to fit our specific architecture, and integrated into our IDE.
Claude (Anthropic)	Claude Sonnet 4.6, free	Assisted with coding SQLi level-hiding logic, file upload vulnerability	Reviewed and tested all generated code against the existing

		module, XSS challenges, and CSRF/IDOR controllers and templates.	project structure. Adapted to our conventions, identified integration issues, and verified end-to-end behaviour through manual testing.

8.2 Appendix - Prompt History Summary

Full prompt histories are maintained in individual progress report appendices submitted throughout the term. Key prompts used during the project included:

- Claude (Sonnet): Bootstrap 5 hero section layout patterns for Thymeleaf homepage.
- Claude (Sonnet): SQLi level-hiding using server-side Thymeleaf th:if conditions and per-level completion flags.
- Claude (Sonnet): FileUploadController with unrestricted MultipartFile endpoint and fileupload.html template structure.
- Claude (Sonnet): CsrfController and IdorController with intentionally vulnerable endpoints, and corresponding Thymeleaf templates.
- Microsoft Copilot: Conversational prompts requesting boilerplate Java code for Spring Boot controllers, data entity structures, and syntax explanations for Thymeleaf templating throughout the term.

9. Work Logs

9.1 Thilan Udahage Don

Date	Hours	Description of Work
Jan 20	1	Participated in instructor-led project consultation. Discussed feasibility of building an intentionally vulnerable Spring Boot application.
Jan 20	1	Conducted research on existing vulnerable web applications (DVWA, WebGoat, Juice Shop). Documented findings for proposal use.
Jan 21	1.5	Co-drafted the pre-proposal for instructor approval. Summarized research findings and outlined initial problem definition and project scope.
Jan 22	1	Reviewed OWASP Top 10 documentation to determine which vulnerabilities to embed into the Spring Boot MVC application.
Jan 23	1	Attended scheduled team meeting. Discussed proposal structure, division of responsibilities, and GitHub setup tasks.
Jan 23	1.5	Created the GitHub repository structure. Added folders (ReportsAndDocuments, Implementation, Misc), created worklog.md and deliverables.md, updated README.
Jan 24	2	Attended Saturday team meeting. Collaborated with Allwyn on drafting the proposal — focused on introduction, research problem framing, and practical contributions.
Jan 25	2	Reviewed proposal for coherence, corrected structural inconsistencies, and improved clarity of research questions.
Jan 26	1	Conducted full review of proposal draft. Verified all sections met rubric requirements. Checked formatting and corrected grammar.
Jan 27	1	Participated in in-class proposal check-in. Added industry evidence from Fortinet's 2024 Cybersecurity Skills Gap Report to strengthen problem motivation.
Jan 31	1	Attended Friday team meeting. Discussed Week 5 tasks including Spring Boot skeleton setup, H2 database configuration, and first vulnerability planning.

Feb 1	1.5	Attended Saturday team meeting. Reviewed architecture outline and refined user flows. Finalized vulnerability placement strategy for SQLi, XSS, and Broken Auth.
Feb 4	1.5	Set up local Spring Boot development environment. Verified dependencies and tested basic application startup.
Feb 5	1	Conducted research on SQL Injection patterns in Spring Boot. Reviewed insecure repository usage examples.
Feb 7	1	Added implementation_plan.md and vulnerability_placement.md to GitHub. Documented planned workflows and frontend responsibilities.
Feb 24	1.5	Redesigned SQL Injection module based on instructor feedback. Planned three-level challenge structure and drafted new UI layout using Bootstrap accordion.
Feb 25	2	Removed old SQL Injection demo logic. Implemented new SqlInjectionController with AJAX endpoints for level validation and progress tracking.
Feb 27	2.5	Implemented SqlInjectionChallengeService with payload validation logic for each level. Added pattern matching for OR, numeric, and boolean SQLi.
Feb 28	2	Added Thymeleaf conditions to lock/unlock levels. Implemented partial UI logic for hiding Level 2 and Level 3.
Mar 3	1	Met with the instructor for a project check-in. She suggested adding a user profile progress tracker. Discussed integration with ChallengeProgress entity.
Mar 4	1.5	Met with Allwyn and conceptualized the 'Why It Worked' Explainer feature — a post-challenge explanation section reinforcing learning outcomes.
Mar 6	1	Emailed the instructor with description and mockup of the 'Why It Worked' Explainer concept. Received approval to proceed.
Mar 7	1.5	Friday team meeting. Reviewed homepage progress, SQLi level-hiding approach, and planned next steps for profile tracker and Explainer.
Mar 8	1.5	Saturday team meeting. Discussed profile tracker design and Explainer structure. Agreed on implementation approach and divided tasks.

Mar 8	2	Worked on homepage redesign and SQLi level-hiding fix — investigated using server-side th:if conditions to fully hide locked levels.
Mar 9	2	Completed homepage and SQLi level-hiding implementation. Updated SqlInjectionController and sqli.html. Tested all three challenge levels end-to-end.
Mar 10	1.5	Began implementing user profile progress tracker. Created profile page template and updated controller to fetch ChallengeProgress data.
Mar 11	2	Continued user profile page. Added Bootstrap badges for completed/incomplete levels and integrated with shared navbar fragment.
Mar 11	1.5	Started 'Why It Worked' Explainer for SQLi Level 1. Added hidden explanation section revealed on successful challenge completion.
Mar 13	1.5	Friday team meeting. Reviewed file upload progress, profile tracker, and Explainer. Discussed Allwyn's broken authentication module.
Mar 14	1.5	Saturday team meeting. Reviewed implementations, identified profile/auth integration issues, and planned final PR4 tasks.
Mar 14	2	Began file upload vulnerability implementation. Set up FileUploadController with no file type validation. Created fileupload.html template.
Mar 15	1	Completed 'Why It Worked' Explainer for SQLi Levels 2 and 3 with JavaScript reveal logic.
Mar 15	1	Applied homepage polish: fixed card spacing, updated button labels, added subtitles to challenge cards.
Mar 16	2	Completed file upload vulnerability. Verified dangerous file types are accepted. Tested Explainer section display.
Mar 17	2	Began CSRF and IDOR module implementation. Researched attack patterns, planned controller design, and polished user profile tracker.
Mar 18	2.5	Implemented CsrfController and IdorController. Created csrf.html and idor.html templates demonstrating each vulnerability. Used Claude to assist.

Mar 20	2	Added 'Why It Worked' Explainer to file upload, broken auth, CSRF, and IDOR modules. Fixed progress tracker persistence bugs.
Mar 21	1.5	Friday team meeting. Reviewed CSRF and IDOR implementations and aligned on final integration tasks.
Mar 22	1.5	Saturday team meeting. Full platform walkthrough. Updated homepage, OWASP reference page, and shared layout navigation.
Mar 23	2	Completed final bug fixes across all modules. Performed end-to-end testing of all challenge modules, Explainer sections, and profile tracker.
Mar 24	2	Applied UI/UX enhancements across the platform. Improved visual consistency of challenge cards, refined spacing and typography on the SQLi and XSS module pages, and updated button styles for better usability.
Mar 25	1.5	Continued UI/UX refinements. Improved mobile responsiveness on the profile tracker page and ensured the navbar fragment renders consistently across all module pages. Committed all changes to GitHub.
Mar 26	1.5	Performed functional testing across all challenge modules. Verified payload validation, Explainer reveal behaviour, and progress persistence for SQLi, XSS, CSRF, and IDOR modules.
Mar 27	2	Friday team meeting with Allwyn. Reviewed remaining tasks before final submission, aligned on report writing responsibilities, and confirmed the UAT test plan.
Mar 28	1.5	Saturday team meeting with Allwyn. Reviewed each other's module implementations, confirmed all challenge levels work end-to-end, and divided final report sections for writing.
Mar 29	1.5	Continued testing — focused on the file upload and broken authentication modules. Identified and fixed a minor UI issue where the success feedback message was overlapping the Explainer section on smaller viewports.
Mar 31	2	Began writing the final report. Drafted the Introduction, Summary, and Changes to Proposal sections.
Apr 1	2	Continued final report writing. Drafted the Implemented Features section for all modules. Added screenshot and code snippet placeholders throughout.

Apr 2	2	Drafted the Evaluation, Reflections, and Concluding Remarks sections. Reviewed and updated the AI Use section to reflect all tools used across the full term.
Apr 4	2	Performed UAT testing across all six challenge modules. Verified all accepted payloads produce correct success responses and Explainer reveals. Logged remaining issues for final fix.
Apr 5	2	Applied final bug fixes identified during UAT testing. Committed all changes to the main branch on GitHub.
Apr 7	2	Updated the GitHub README with project description, installation instructions. Verified the repository structure is clean and committed the final check-in.
Apr 8	2	Reviewed and rehearsed the presentation slides. Confirmed demo responsibilities with Allwyn and prepared talking points for the defense Q&A.

9.2 Allwyn Mascarenhas

Date	Number of Hours	Description of Work
15th Jan	2	Conducted initial research on OWASP Top 10 vulnerabilities and reviewed documentation for DVWA, WebGoat, and Juice Shop. Compared their learning approaches and identified opportunities for a simpler, framework-based platform.
20th Jan	1	Participated in instructor-led project consultation. Discussed project direction, technology stack, and how to embed vulnerabilities safely within a Spring Boot MVC application.
21st Jan	1.5	Co-drafted the pre-proposal for instructor approval. Summarized research findings and outlined the initial problem definition, research questions, and project scope.
22nd Jan	1	Researched Spring Boot security misconfigurations and common vulnerability patterns (e.g., insecure authentication flows, improper input validation). Documented potential implementation strategies.

Date	Number of Hours	Description of Work
23rd Jan	1	Attended scheduled Friday team meeting. Finalized task distribution and reviewed proposal template.
23rd Jan	1.5	Created and organized the GitHub Project Kanban board. Added tasks for proposal writing, research, vulnerability implementation, and documentation. Assigned responsibilities and set up workflow columns.
24th Jan	2	Collaborated with Thilan on drafting the full project proposal. Wrote sections on scope, methodology, and technology stack. Ensured the proposal met the rubric requirements for introduction and research design.
25th Jan	2	Continued refining the proposal independently. Reviewed the draft for clarity, strengthened the methodology section, and added additional justification for the selected technology stack.
26th Jan	1	Reviewed and refined the proposal draft. Added details to the methodology and expected results sections. Submitted the document to Blackboard and GitHub.
27th Jan	1	Participated in in-class proposal check-in. Instructor requested adding industry data to justify the project. Reviewed Fortinet's 2024 Skills Gap Report and incorporated the statistic that 58% of IT decision-makers cite lack of hands-on skills as a top cause of breaches.
31st Jan	1	Attended scheduled Friday team meeting. Discussed Week 5 implementation tasks, including setting up the Spring Boot skeleton and configuring the H2 database.
1st Feb	1.5	Attended scheduled Saturday team meeting. Reviewed architecture outline and refined backend workflow diagrams. Finalized vulnerability placement strategy for SQL Injection, Broken Authentication, and IDOR.
3rd Feb	1	Began setting up backend project structure. Created initial controller and service templates to prepare for SQL Injection implementation.

Date	Number of Hours	Description of Work
4th Feb	1.5	Completed Spring Boot skeleton setup. Verified project builds successfully, configured H2 database, and tested basic backend routing.
5th Feb	1	Conducted deeper research on SQL Injection exploitation in Java applications. Reviewed insecure query patterns and prepared notes for implementing the first vulnerability.
6th Feb	1	Drafted backend-focused content for Progress Report 1, summarizing architecture planning, backend setup, and SQL Injection research.
7th Feb	1	Updated backend folder structure and added initial placeholder files for SQL Injection implementation. Reviewed vulnerability placement documentation added by Thilan.
9th Feb	2	Implemented the vulnerable SQL query using string concatenation. Verified that SQLi payloads produced unsafe behavior and returned unintended results.
10th Feb	1.5	Implemented the safe SQL query using prepared statements. Ensured parameterized queries blocked injection attempts and added backend logic to toggle between safe and vulnerable modes.
13th Feb	1	Weekly team meeting. Reviewed SQLi progress, discussed RXSS routing improvements, and planned controller refactoring for better modularity.
14th Feb	1	Continued team meeting. Finalized SQLi backend logic and confirmed the structure for GET-based reflection in the upcoming RXSS controller.
15th Feb	1	Added backend support for executed SQL query display and improved error handling for malformed SQL payloads. Ensured the application returned meaningful error messages without breaking.
17th Feb	1	Created a dedicated RxssController to separate XSS logic from other modules. Cleaned up routing and ensured proper GET-based reflection behavior.

Date	Number of Hours	Description of Work
18th Feb	1	Adjusted the CSP header from Content-Security-Policy to Content-Security-Policy-Report-Only to allow XSS payload execution for demonstration while still logging violations.
20th Feb	1	Performed integration testing for SQLi and RXSS modules. Verified backend compatibility with UI components, dropdown behavior, and scroll helper functionality.
21st Feb	1.5	Wrote backend sections of SQLi implementation details and RXSS controller explanation. Participated in recording the midterm demonstration video. Uploaded updated code, the completed midterm report, and the demonstration video to the GitHub repository.
22nd Feb	2	Designing and coding the student tracking page.
24th Feb	2	Coding layout for user profile page
26th Feb	2	Creating and testing user profile controller, css, html.
1st March	2	Reading and research for broken auth vuln deployment.
3rd March	2	Student tracking background coding
5th March	1	UI redesign call with team member to create wire frames
10th March	2	Reviewing owasp broken auth literature
12th March	2	Wireframing and coding for broken auth learning modules
13th March	3	Progress report 4 drafting and review
18th March	2	Remote team meeting to finalize final design decisions and UAT test cases.
20th March	2	UAT test case designed for final testing and submisson
21st March	3	UI improvements and vuln module improvement code changes made and committed.

Date	Number of Hours	Description of Work
26th March	3	UAT testing discussion and tests writeup
1st April	2	UAT testing with team
2nd April	2	Final report write up and editing
3rd April	2	Cloud hosting research and deployment.
4th April	2	Performed UAT testing with Thilan. Focused on broken authentication and IDOR modules. Verified all challenge levels, progress tracking, and Explainer sections function correctly end-to-end.
5th April	2	Successfully deployed the application to Google Cloud Run. Configured the containerized Spring Boot application, set up the Cloud Run service, and verified the platform is accessible via the public URL. Updated the README with the live deployment link.
7th April	2	Updated the GitHub repository — ensured all reports and documents are checked into the main branch under ReportsAndDocuments. Verified the final codebase is committed, cleaned up stale branches, and confirmed the README reflects the final deployed state of the project.
8th April	2	Reviewed and rehearsed the presentation slides. Confirmed demo responsibilities with Thilan and prepared responses for anticipated defense questions, particularly around broken authentication and the Google Cloud Run deployment.

10. Concluding Remarks

Insecure by Design successfully delivers a hands-on web security training platform that addresses a genuine gap in existing cybersecurity education tools. By combining progressive challenge levels with the 'Why It Worked' Explainer feature, the platform moves beyond passive demonstration and actively reinforces learner understanding of attack mechanisms. The six implemented vulnerability modules SQL Injection, XSS, CSRF, IDOR, Broken Authentication, and File Upload; provide broad OWASP Top 10 coverage within a modern, accessible Spring Boot application.

The project demonstrated the value of iterative development and responsiveness to feedback. The mid-term pivot to a challenge-based architecture, the addition of the Explainer feature, and the expansion of the module set all emerged from ongoing dialogue with the instructor and user testers, producing a substantially stronger final product than originally proposed.

Future work could extend the platform with additional vulnerability types, user account management, a scoring or leaderboard system, and instructor-facing analytics. The 'Why It Worked' Explainer framework is sufficiently modular to be extended to any new challenge module with minimal additional development effort. The platform represents a solid foundation for an open-source cybersecurity education tool with meaningful real-world applicability.

11. References

Fortinet. (2024). 2024 Cybersecurity Skills Gap Report. Retrieved January 2026, from https://www.fortinet.com/content/dam/fortinet/assets/reports/2024-cybersecurity-skills-gap-report.pdf

Mudge, R. (digininja). (n.d.). Damn Vulnerable Web Application (DVWA) [GitHub repository]. Retrieved January 2026, from https://github.com/digininja/DVWA

OWASP Foundation. (n.d.). OWASP Juice Shop. Retrieved January 2026, from https://owasp.org/www-project-juice-shop/

OWASP Foundation. (n.d.). OWASP WebGoat. Retrieved January 2026, from https://owasp.org/www-project-webgoat/

OWASP Foundation. (n.d.). OWASP Top Ten. Retrieved March 2026, from https://owasp.org/www-project-top-ten/

TryHackMe. (n.d.). TryHackMe | Cyber Security Training. Retrieved March 2026, from https://tryhackme.com/

Spring. (n.d.). Spring Boot Reference Documentation. Retrieved February 2026, from https://docs.spring.io/spring-boot/docs/current/reference/html/

Appendix A: Installation Guide

Live Deployment Access

For immediate access without local installation, the fully functional platform is hosted live on Google Cloud Run and can be accessed at: https://ibd-1068343979302.us-central1.run.app/

Local Installation Instructions

This project is provided as a **ZIP file** inside the repository under: /Implementation

1. Download the ZIP File

1. Navigate to the Implementation folder in the repository
2. Download the idb ZIP file
3. Extract it anywhere on your computer

2. Open IntelliJ IDEA

- Launch IntelliJ IDEA
- Click **File → Open**
- Select the extracted project folder
- IntelliJ will automatically detect it as a **Maven** Spring Boot project
- Wait for IntelliJ to finish indexing and downloading dependencies (this happens automatically)

3. Run the Spring Boot Application

1. In the IntelliJ Project Explorer, open: \ibd\src\main\java\com\akmlab\ibd\IbdApplication.java
2. Click the **green Run button** next to the main method
3. IntelliJ will start the embedded server (Tomcat)
4. Once running, you will see: Started IbdApplication in X seconds in the log at the bottom

This means the application is ready.

Accessing the Demo

Open your browser and go to: http://localhost:8081

User Account

Username: allwyn Password: akmlab

Appendix B: User Guide

The following guide describes how to use the Insecure by Design platform as a learner.

Getting Started

After launching the application and navigating to http://localhost:8080, you will see the homepage with challenge cards for each available vulnerability module. Click any challenge card to begin that module.

Completing a Challenge

- Read the challenge description to understand the scenario and objective.
- Enter a payload in the input field and submit the form.
- If your payload is incorrect, you will receive a feedback message explaining what went wrong.
- If your payload is correct, the level will be marked as complete, and the next level will be unlocked.
- After completing a level, the 'Why It Worked' Explainer will appear, explaining the attack mechanism in detail.

Tracking Your Progress

Navigate to your profile page via the navbar to see your completion status for all challenge modules. Completed levels are indicated with a green badge and incomplete levels with a grey badge.

Challenge Modules and Accepted Payloads

- SQL Injection — Level 1: ' OR '1'='1 | Level 2: 1 OR 1=1 | Level 3: 1 AND 1=1
- XSS — Level 1: <script>alert(1)</script> | Level 2: <img src=x onerror=alert(1)> | Level 3: " onmouseover="alert(1)
- CSRF — Level 1: Any attacker@domain.com email | Level 2: csrf_token_123
- IDOR — Level 1: Any ID from 1–5 | Level 2: ID = 1 (admin)
- Broken Auth — Level 1: admin/admin | Level 2: admin or user-1 or YWRtaW4= | Level 3: john.doe@example.com/password123
- File Upload — Level 1: Upload payload.html with <script> content | Level 2: Upload evil.jpg with HTML content | Level 3: Upload payload.html.jpg

OWASP Reference

The OWASP page (accessible via the navbar) links each implemented challenge module to its corresponding OWASP Top 10 category, providing additional reading and real-world context for each vulnerability type.